Verified Context Retirement: Safety Conditions and Retrieval–Memory Trade-offs

Author Names, *Member, IEEE*

***Abstract*—When can an agent stop remembering its execution history without losing the current evidence and obligations it needs to finish correctly—and in which cost regime might the answer pay? We study this question at the *mechanism level*: a pre-registered, factorial, frozen-protocol simulator study of context retirement, with 1,176 runs crossing seven retention policies with a scheduler factor, plus a 160-episode long-horizon sub-cohort, across four registry sizes, three dependency families, and seven seeds, with temporal persistence, obligation lifetimes, three-phase accounting, a five-episode capsule audit against stream ground truth, and two frozen cost regimes (bytes; bytes plus a per-round-trip constant). Within this simulator, the pattern that survives is a *conversion heuristic with a regime-conditional payoff*. Every measured policy that retires history through a lossy summary fails the acceptance contract (periodic summary/reset accepts 28 of 84 runs, with 109,761 stale reads and 385 lost obligations), while the policy that first converts history into verified interface bindings, a typed pending-obligation set, and declared dependencies accepts 84/84 and passes the capsule audit with zero defective service. The payoff direction in the simulator is operations, not bytes: frontier retirement cuts on-demand retrieval operations by 44.0% pooled (45.1% at four times the horizon; sign test $p{=}10^{-25}$ on round-trip-regime cost) and is cheaper than masking on bytes by 8.0% pooled—below the frozen 20% byte gate, so no byte-threshold win is claimed. A closed-form break-even rule (Proposition 4) converts the frozen sensitivity sweep into a conditional threshold formula ($c_{0.8} \approx 1{,}364$ B pooled in the simulator); it is a property of the frozen accounting, not validated deployment guidance. Sufficiency of the conversion heuristic is proved *within the model* (Proposition 1): under the engine's stream invariants and reliable retrieval, the frontier policy provably incurs zero staleness and zero obligation loss—a structural prediction of the simulator, not a guarantee about agents. The scheduler factor is byte-neutral by design and measured at 0.000% leak with an 87.5% makespan reduction. Grouped certificate layouts survive only as a conditional component selected by measured update density (dense stratum pass; 0.0% decision-rule regret), and a demand-driven adaptive-coordination variant was evaluated against pre-registered promotion gates and *rejected* (−0.0% versus frontier). Pre-specified exploratory analyses (labeled as such) stress the result: a bounded-recency comparator is cheaper than frontier below a stated break-even price but leaves 69.0% of due obligations outside its retention, quantifying what it delegates to the retrieval layer; and a replay of a real repository's change history through the same engine preserves the operations advantage (23.0% cut) with the density rule matching the oracle on its first real trace. All experiments are simulator-level: no hosted model was called, no token or billing claim is made, and the live-agent cohort—which measures retrieval, context, acceptance, and end-to-end outcomes under matched budgets—remains future work with its protocol pre-registered here.**

***Index Terms*—Bounded evidence, verification contracts, bandwidth-limited logging, completion hyperedges, cost-aware layout selection.**

This artifact candidacy draft accompanies the `future-code` research repository. Author names and affiliations require confirmation before any submission. Mathematical development, implementation, experiments, and writing were AI-assisted. This is a research artifact, not an accepted or peer-reviewed venue. (Version 8.0, September 28, 2026: major revision with scoped claims and consistency audit.)

I. Productivity Boundary and Research Scope

The governing research objective is to reduce the cost and time required for agents to complete large software projects correctly, without increasing coordination failures or weakening acceptance contracts. That objective outruns what any single paper can measure, and this manuscript is explicit about the ladder of evidence it occupies.

a) What this paper is.: It is a simulator study of context retirement (Section IV) plus the verification and evidence components the surviving heuristic depends on (Section VI, with derivations in the appendices). Two protocol generations were run; the current one is factorial, pre-registered, and git-frozen before data generation. Five closure rules were declared before data; two of the paper's own proposed extensions failed their gates and are reported as rejected: the byte-regime productivity claim for frontier retirement (it saves only 8.0% on submitted bytes versus masking—below the 20% gate, no win claimed) and the demand-driven adaptive-coordination variant (−0.0% regime-O versus plain frontier; promotion gate ≥10%). What survives is (i) a conversion heuristic supported by the simulator's evidence—in the frozen arm list, every policy that retires history through a lossy summary fails the acceptance contract at every scale tested (28/84 at the largest grid), while the policy that first converts history into verified bindings, typed obligations, and declared dependencies passes it; (ii) an operations result at equal correctness—frontier retirement cuts on-demand retrieval operations 44.0% and round-trip-regime cost 29.6% versus observation masking, persisting at 4× horizon, with a clean capsule audit; and (iii) a density-selected conditional role for certificate-tree evidence layouts (0.0% decision-rule regret). HACT is not the paper's center; it survives only where its ablation says it wins.

b) The boundary that remains.: The retirement study is a simulator study. It models information flow between

a context policy and a seeded work stream; it does not call a hosted model, and it cannot establish that any agent finishes faster, bills less, or reasons better. Its staleness, obligation-loss, and audit results are structural properties of policies in that simulator, not measurements of agents. The live-agent cohort that could test those predictions has its protocol pre-registered in Section IX-A0b and has not been run. The negative controls below—flat checker timings, per-episode verification overhead, unchanged root-status inputs—are reported as limits of the serialization claim, and the retirement cohort's own closures are reported as limits of the mechanism claim. Nothing in this paper is an agent-productivity measurement, and the abstract says so.

II. Introduction

Long-horizon software agents accumulate execution history, and the accumulation is not free: resubmitting it costs bytes that grow with project length, and naively discarding it costs correctness. The research question of this paper is the review's stronger form:

> When can an agent stop remembering its execution history without losing the current evidence and obligations needed to finish correctly—and in which cost regime does the answer pay?

We answer it with a pre-registered, factorial mechanism study (Section IV). Seven retention policies—from full replay through observation masking, periodic summary/reset, bounded pages, and a verified board, to verified-frontier retirement with typed capsules, plus a demand-driven adaptive variant—are crossed with a scheduler factor in a full factorial, and consume byte-identical seeded work streams with temporal persistence, payload-bearing facts, persistent dependency footprints, and obligations that come due later. Accounting is split into construction, steady state, and a five-episode audit tail that compares each policy's capsule against stream ground truth; two cost regimes are frozen ex ante, one byte-bound and one round-trip-bound. The v2 protocol and its five closure rules were committed to git before any v2 data was generated; the cohort and its tables were committed before any v2 manuscript text was written. Two protocol generations were run; the first's design limitations are stated in Section IV-B and its full record is retained in Appendix J.

The pattern that survives both generations in this simulator is a conversion heuristic with a regime-conditional payoff. *Failure direction (measured)*: every policy that converts lossily—periodic summary/reset—fails the frozen acceptance contract at every scale measured (28/84 runs at the v2 scale; 21/63 at the v1 scale), silently reverting values to beliefs older than the working window and dropping obligations that cross reset boundaries; every measured policy that first converts history into verified interface bindings, a typed pending-obligation set, and declared dependencies passes it. *Payoff*: the payoff direction is operations, not bytes—but the byte direction is also positive. Frontier retirement is cheaper than masking on bytes by 8.0% pooled (76 of 84 cells, p=5×10^{-15}; below the frozen 20% byte gate) and on round-trip-regime cost by 29.6% (84 of 84 cells, p=10^{-25}), cutting operations 44.0% versus masking with the advantage persisting at four times the horizon (45.1%). A closed-form break-even rule (Proposition 4) converts the frozen sensitivity sweep into a conditional threshold formula: the simulator's gate flips at round-trip price $c_{0.8} \approx 1{,}364$ B pooled on the frozen cohort—a property of the frozen accounting, not validated deployment guidance. The live-agent hypothesis this scopes is precise: savings, if any, come from avoided tool-call round trips; the byte direction is a bonus too small to carry the claim. *Audit*: the promoted mechanism passes a capsule audit against stream ground truth with zero defective service, and the audit's development history includes a disclosed, test-caught reference fix (Section IV-D). *Sufficiency (within the model)*: the conversion heuristic is provably sufficient under the engine's semantics (Proposition 1): given the stream invariants and reliable retrieval, the frontier policy incurs zero staleness and zero obligation loss. This is a model-internal guarantee—a structural prediction of the simulator—not a claim about agents. *Stress*: pre-specified exploratory analyses (Section IV-E) put the result under load: a bounded-recency window beats frontier below a break-even price of 24,444 B but leaves 69.0% of due obligations outside its retention; a real repository trace replayed through the engine preserves the operations cut (23.0%) with the density rule matching the oracle; and the per-seed range of the operations cut never crosses zero. *Rejected extensions*: a demand-driven adaptive-coordination variant was evaluated against a pre-registered promotion gate and rejected (−0.0% regime-O versus plain frontier; the gate required ≥10%), and the HACT certificate-tree evidence layout remains a conditional component—against a steel-manned flat ledger it wins dense updates and loses sparse ones, so it is selected by a measured-density rule with 0.0% regret rather than by default. The scheduler factor is cleanly separated: measured byte leak 0.000%, makespan reduction 87.5%.

Section VI condenses the verification and evidence components the conversion heuristic relies on, unchanged in substance: immutable identities, packet cost models, layout learning, and the negative controls that bound what serialized evidence can claim; its full derivations, including the exact-DP reference formulation, now live in the appendices. A capsule that carries verified bindings and pending obligations needs an evidence layout when its contents must cross a link, and the layout's value is conditional in exactly the way the ablation measures. Sections VII–IX bound the delivery, adoption, and trust claims.

The negative controls remain: flat checker timings and per-episode verification overhead are limits of the serialization claim; the mechanism-level cohort is a limit of the retirement claim; the live-agent protocol of Section IX-A0b has not been run, and no hosted model, token count,

or billing figure appears anywhere in this paper as a measurement. Executing that pre-registered protocol is the study's highest-priority next experiment.

III. Related Work and Positioning

a) Context management for long-horizon agents.: Observation masking with focused retrieval is the strongest simple baseline for bounded context [16]; context folding [17] branches and folds trajectories; periodic summary/reset is the generic compaction baseline. Our mechanism study steel-mans all three in a factorial design and reports where each fails: masking is near-optimal on submitted bytes but pays for every non-current read with a retrieval operation (44.0% more operations than frontier retirement at equal correctness), while summary/reset — cheapest on bytes—fails the acceptance contract on 56 of 84 runs through silent staleness and lost obligations. This locates our contribution precisely: not a better compaction heuristic, but a measured characterization of *which retained structures* (typed pending obligations, current verified bindings, dependency interfaces) make forgetting safe, and a pre-registered operations metric for the live cohort that could confirm it.

b) Multi-agent coordination.: Decentralized shared context [19] and adaptive communication topologies address central-manager bottlenecks; controlled scaling studies [18] show coordination can hurt sequential or tool-heavy work. Our verified-board arm is exactly such a baseline and holds correctness (84/84) at the cost of shipping a full present-state row per gate; the scheduler factor is orthogonal and byte-neutral by design (measured leak 0.000%), consistent with the task-architecture alignment those studies report.

c) Testing, impact analysis, and incremental computation.: Test prioritization seeks earlier fault exposure [1]; regression selection uses change dependencies [2], [3]. Our dependency footprints and completion records inherit that lineage; the certificate tree is an incremental-publication structure, not a selection structure, and its new ablation shows its value is conditional on update density (dense global updates favor the tree by 91.3% at the v2 scale; sparse deltas favor a flat ledger).

d) Benchmarks.: SWE-bench [14] and its harness [21] provide the external anchor the live cohort pre-registers in Section IX-A0b; no benchmark result appears in this paper because no live agent was run.

e) What is not claimed.: Serialization savings are not compression savings: fewer padded bytes need not imply fewer compressed bytes, and we measure both where both exist. Flat ledgers and a trusted root-status endpoint can be much smaller than an archive of aggregation updates; they satisfy different delivery needs and are steel-manned, not hidden. No claim in this paper extends to agent speed, billing, or reasoning quality; those belong to the unrun live cohort, whose protocol is pre-registered with closure rules.

IV. When Can an Agent Stop Remembering? A Pre-Registered Mechanism Study

A. The question

Long-horizon agents face a retention decision every episode: resubmit history, mask it and re-fetch on demand, or summarize it away. The research question is when the third option—*retirement*—is safe, and what it buys:

> *When can an agent stop remembering its execution history without losing the current evidence and obligations it needs to finish correctly, and in which cost regime does the answer pay?*

The study is *mechanism-level*: every number below is produced by a deterministic engine that models information flow between a context policy and a seeded work stream. No hosted model is called. The pattern that survives two frozen protocol generations in this simulator is a **conversion heuristic with a regime-conditional payoff**:

> *In this simulator, every measured policy that retires history through a lossy summary fails the acceptance contract, while the policy that first converts history into (i) verified interface bindings for every fact another component will read, (ii) a typed pending-obligation set, and (iii) declared dependency interfaces passes it. The payoff direction is not bytes: it is avoided retrieval round trips, which pay only in the round-trip-bound regime.*

B. Study provenance and what changed between generations

Two protocol generations were run, both pre-registered.

a) Generation v1 (file-frozen 2026-09-23).: Ten hand-picked arms over $3 \times 3 \times 7 = 63$ cells (630 runs) with three closure rules. Its closures fired: no byte-cost win for frontier retirement over masking (null closure), an attribution closure that forced the dimension-separated reading, and a conditional pass for grouped certificate layouts. v1's protocol and engine were frozen in files before its cohort ran, but were committed to git only on 2026-09-24, after the v2 protocol commit; v1's freeze is therefore file-attested only, and this ordering is disclosed here rather than smoothed over. The full v1 design, its numbers, and its deviations remain in Appendix J.

b) Generation v2 (git-frozen 2026-09-24, commit before data).: v1 answered "what does each policy lose" but had four design weaknesses: a hand-picked arm list rather than a factorial, a single pooled byte threshold, capsule accounting evaluated only as an end-of-run total, and one cost regime. v2 repairs all four ex ante: (1) a full factorial over retention R (7 levels), scheduler S (2 levels), with factor contrasts replacing pairwise arm comparisons; (2) three-evaluation separation—every run's accounting is split into construction (episodes 0–4), steady state (5–34), and a five-episode audit tail (35–39) in which an independent auditor compares the policy's capsule against stream ground truth (A1 stale retention, A2 pending-set mismatch, A3 defective

service); (3) two frozen cost regimes—bandwidth-bound B (bytes) and round-trip-bound O (bytes +4,096 B per retrieval operation, with sensitivity at 1,024 and 16,384 B); (4) scale: four registry sizes ($n \in \{128, 512, 1{,}024, 4{,}096\}$), three families, seven seeds, forty episodes ($4 \times 3 \times 7 = 84$ cells $\times 14$ policy runs = 1,176 runs), plus a long-horizon sub-cohort ($n = 1{,}024$, 160 episodes, 63 runs). Five closure rules were frozen before data generation; the v2 cohort and its tables were committed before any v2 manuscript text was written. One frozen-document miscount ("42 runs" for a grid the protocol itself defines as 63) was corrected ex ante in the engine notes and is disclosed here.

C. The frozen arms and closure rules

a) Retention levels.: (1) `full-replay`: resubmit all history. (2) `masking`: identity-only context, every non-current read fetched on demand—the strongest simple baseline. (3) `summary-reset`: periodic summary/reset every 4 episodes with a window budget. (4) `bounded-hier`: bounded pages. (5) `verified-board`: a present-state row per known gate. (6) `frontier`: verified-frontier retirement—retain only the working set, boundary neighbors, dependency footprint, and pending obligations, as a typed capsule (160 B base, 16 B per retained binding actually held, 48 B per pending obligation, 64 B aggregate invariants). (7) `frontier-adaptive`: frontier plus a frozen demand-driven eviction rule (bindings unused for 8 episodes and opened $\geq$8 episodes ago are evicted outside the exempt base)—the *adaptive-coordination* candidate.

b) Closure rules (all frozen ex ante).: Null closure: frontier is promotable only if it cuts regime-O cost per accepted run by $\geq$20% versus masking pooled, with no family stratum regressing by more than 10%. *Attribution closure*: the scheduler factor is byte-neutral by design; a byte leak exceeding 1% of the grand mean voids all scheduler results. *HACT closure*: grouped layouts are promotable only as a conditional component—a $\geq$5% win in the dense stratum plus a stated density decision rule with reported regret. *Adaptive-coordination closure*: `frontier-adaptive` is promotable only at equal acceptance, zero audit failure, $\geq$10% regime-O reduction versus frontier, and no family regression above 5%. *Audit closure*: any arm with A3 defects in its audit tail is reported as audit-failing and cannot be promoted.

D. Results

a) Lossy conversion fails; typed conversion does not.: Periodic summary/reset—the cheapest policy on submitted bytes—accepts only 28/84 runs: 109,761 stale reads and 385 lost obligations at the new scale, the same structural failure v1 measured (30,820/280 on its smaller grid). The failure mechanism is exact: a summary reverts values to a belief older than the working window, and obligations crossing a reset boundary survive only through a lossy summary. Every policy that converts before retiring—frontier, and at higher cost the verified board and full replay—accepts 84/84 with zero staleness and zero loss. Bounded pages accept 84/84 but pay 1,211 ops/run (896 in the steady phase); masking accepts 84/84 and pays 1,875 ops/run (1,436 steady). Within the frozen arm list this is the observed separation; it is evidence for the conversion heuristic, and Proposition 1 supplies the complementary model-internal guarantee.

b) The null closure passes in the round-trip regime, not the byte regime—exactly as pre-registered.: Frontier retirement reduces regime-O cost per accepted run by 29.6% versus masking (gate $\geq$20%), with *no* family stratum regressing (worst stratum: none). In the bandwidth regime the same comparison is only 8.0%—we repeat v1's honest negative and claim no byte-cost win. The two regimes separate because frontier's advantage is *operations*: 44.0% fewer retrieval round trips per accepted run pooled (66.5% on clustered, 43.0% on independent; global coupling leaves no gap because every read is a current fact for both policies). Sensitivity across the frozen round-trip constant: 17.8% at 1,024 B, 29.6% at 4,096 B, 38.8% at 16,384 B—the advantage grows with the round-trip price and is positive across the entire pre-registered sensitivity range.

c) The audit tail is clean for the promoted mechanism.: Across all 1,176 runs, the five-episode capsule audit records zero A3 defective service for every arm (audit closure: **clean**): retained bindings match stream generations, pending sets match the stream's true pending sets in both directions, and no required read was served from defective state. The audit also caught a reference bug during development—the ground-truth pending set originally deduplicated obligation openings by first rather than latest opening, misclassifying correctly-held re-opened obligations as phantoms—which is disclosed in the repository history and fixed before the frozen cohort was regenerated.

d) Adaptive coordination was evaluated and rejected.: The demand-driven eviction variant (`frontier-adaptive`) matches frontier's acceptance (84/84) and audit cleanliness, but its regime-O reduction versus plain frontier is -0.0% (promotion gate $\geq$10%): the frozen use-clock rule evicts almost nothing the base frontier does not already evict, because the base exemptions (working set, obligations, footprint) already cover nearly everything the use clock would keep. Per the frozen closure it is **not promotable**, and the manuscript's mechanism carries no adaptive component. This is the paper's second honest negative after v1's byte-regime null result, and it was decided by a pre-registered gate, not by narrative preference.

e) The scheduler factor is cleanly separated.: The full factorial confirms byte-neutrality by design: the scheduler main effect on regime-B bytes is 0.000% of the grand mean (leak threshold 1%; attribution closure **not triggered**), while the makespan interaction is large—adaptive batching cuts makespan rounds by 87.5% (24,471 to 3,070 rounds/run pooled). Factor contrasts, not arm pairs, carry this claim: every $R \times S$ combination was run in every cell.

TABLE I

FROZEN V2 COHORT AT S = fixed: 84 CELLS × 7 RETENTION POLICIES. REGIME-B BYTES AND REGIME-O COST ARE PER ACCEPTED RUN; OPS ARE ON-DEMAND RETRIEVAL OPERATIONS PER ACCEPTED RUN; AUDIT DEFECTS (A3), STALENESS DEFECTS, AND OBLIGATION LOSSES ARE TOTALS OVER THE 84 RUNS OF EACH POLICY. ACCEPTANCE IS THE FROZEN CONTRACT: ANY STALENESS DEFECT OR OBLIGATION LOSS REJECTS THE RUN.

Policy	Accepted	Regime-B B/run	Regime-O B/run	Ops/run	A3	Stale	Lost
full-replay	84/84	12,007,957	12,007,957	0	0	0	0
masking	84/84	5,107,068	12,786,190	1,875	0	0	0
summary-reset	28/84	220,410	220,410	0	0	109,761	385
bounded-hier	84/84	789,534	5,748,522	1,211	0	0	0
verified-board	84/84	12,929,557	12,929,557	0	0	0	0
frontier	84/84	4,698,273	9,002,292	1,051	0	0	0
frontier-adaptive	84/84	4,698,350	9,005,733	1,052	0	0	0

TABLE II

FAMILY STRATA AT S = fixed: REGIME-B BYTES AND REGIME-O COST PER ACCEPTED RUN, AND RETRIEVAL OPERATIONS PER ACCEPTED RUN, MASKING VERSUS FRONTIER. GLOBAL COUPLING LEAVES NO OPERATIONS GAP (EVERY READ IS A CURRENT FACT); THE OPERATIONS ADVANTAGE CONCENTRATES WHERE READS CROSS THE FRONTIER.

Family	Masking B/run	Frontier B/run	Regime-O cut	Masking ops	Frontier ops
clustered	705,448	676,449	39.8%	230	77
independent	2,607,799	2,323,054	39.6%	5,394	3,075
global	12,007,957	11,095,317	7.6%	0	0

f) The operations advantage persists at four times the horizon.: In the long-horizon sub-cohort ($n = 1{,}024$, 160 episodes), frontier cuts retrieval operations by 45.1% and regime-O cost by 30.9% versus masking at equal acceptance (63/63), while regime-B bytes remain within a few percent. The advantage does not decay with horizon. The clustered stratum's regime-O reduction widens (39.8% at the short horizon to 42.5% at the long horizon) even though its raw operations cut narrows (66.5% to 61.9%): at the longer horizon a larger share of frontier's savings sits in avoided bytes per fetch, and the pooled cut widens to 45.1%.

g) HACT remains a conditional component, now with a decision rule that costs nothing.: The evidence-layout ablation (held-out half, order learned on the training half, flat steel-man at identical record cost) reproduces v1's shape at the new scale: the learned tree wins the dense global stratum (169,408 vs 1,936,704 held-out B/cell, a 91.3% cut) and loses the sparse strata. The frozen density rule—select the tree iff measured training-half update density $\geq$0.5—matches the per-cell oracle on every one of the 84 cells (0.0% regret). HACT therefore stays what v1's closure made it: a conditional serialization component with an observable selection signal, not the paper's center. Section VI retains the component development, condensed, with its scope labels unchanged; the full derivations now live in the appendices.

E. Pre-specified exploratory analyses (post-data, labeled as such)

The five closures are the confirmatory results. The analyses in this subsection were specified and committed *after* the cohort was frozen but *before* they were run (`docs/protocols/EXPLORATORY_ANALYSES_v2.md`); they are exploratory and every number below is labeled as such. One amendment (E6, the bounded-hier comparator) was itself prompted by observing E1's pre-specified output; its motivation is disclosed in the protocol document and summarized here.

a) E1/E5: the sensitivity sweep becomes a conditional threshold formula.: Proposition 4 (Section V) replaces the three-point sensitivity with a closed-form break-even: with per-accepted means (B_f, B_m, o_f, o_m), frontier satisfies the 20% gate at round-trip price c iff $c \geq c_{0.8}$, where $c_{0.8} = (0.8B_m - B_f)/(o_f - 0.8o_m)$ whenever the ratio is decreasing and the gate is not already met at $c = 0$. On the frozen cohort the byte direction alone does *not* clear the gate: frontier is 8.0% cheaper on bytes pooled ($R(0) = 0.920$), so $c_{0.85} \approx 658\,\mathrm{B}$ and $c_{0.8} \approx 1{,}364\,\mathrm{B}$ pooled, with every additional unit of round-trip price widening the advantage (pooled ratio $0.920 \to 0.588$ as c goes $0 \to 32{,}768\,\mathrm{B}$). Per family the thresholds are sharper where reads cross the frontier often: $c_{0.8} \approx 1{,}047\,\mathrm{B}$ on clustered, $\approx 191\,\mathrm{B}$ on independent, and no finite price clears the gate in the global family, where both policies are fetch-free and the ratio is constant at 0.924. The frozen $4{,}096\,\mathrm{B}$ price sits above every pooled and per-family threshold it can clear, which is why the frozen null closure fired on regime O; the v1 20% byte gate never fired in v2 because v2's null closure was pre-registered on regime O, not regime B. These thresholds are properties of the frozen accounting: they say where the simulator's gate flips, and the live cohort is the instrument that would measure a real deployment's byte costs, operation mix, and round-trip price.

b) E2: distributional robustness.: Paired over the 84 cells at S = fixed: frontier's regime-O cost is below masking's in 84/84 cells ($p = 1.0 \times 10^{-25}$) and its regime-B bytes below masking's in 76/84 cells ($p = 5.0 \times 10^{-15}$) (exact two-sided sign tests). Per-seed ops reduction ranges

TABLE III
LONG-HORIZON SUB-COHORT ($n = 1{,}024$, 160 EPISODES, SEVEN SEEDS, S = fixed). OPS AND COSTS ARE PER ACCEPTED RUN. THE OPERATIONS ADVANTAGE PERSISTS AT 4× HORIZON; BYTES STAY COMPARABLE.

Family	Policy	Accepted	Ops/run	Regime-O B/run	Regime-B B/run
clustered	masking	7/7	745	4,449,648	1,398,713
clustered	frontier	7/7	284	2,559,028	1,394,594
clustered	frontier-adaptive	7/7	287	2,569,097	1,394,715
independent	masking	7/7	16,051	72,702,403	6,956,921
independent	frontier	7/7	8,943	42,826,402	6,194,704
independent	frontier-adaptive	7/7	8,958	42,888,791	6,196,238
global	masking	7/7	0	34,003,520	34,003,520
global	frontier	7/7	0	31,417,920	31,417,920
global	frontier-adaptive	7/7	0	31,417,920	31,417,920

TABLE IV
EVIDENCE-LAYOUT ABLATION BY FAMILY (POOLED OVER SIZES AND SEEDS; HELD-OUT HALF). THE DENSITY RULE PICKS THE TREE EXACTLY WHEN IT WINS.

Family	Flat B/cell	HACT B/cell	Tree delta
clustered	150,386	204,631	36.1%
independent	277,129	344,823	24.4%
global	1,936,704	169,408	-91.3%

33.8%–50.6% and per-seed regime-O reduction 23.3%–33.7% across the seven seeds: no seed reverses either direction.

c) E6: the bounded-recency comparator, analyzed head-on.: E1's table exposed that `bounded-hier`—a 64-gate recency window—is cheaper than frontier at the frozen 4,096 B price in three of four strata (bytes 0.168× pooled), and the frozen closure never compared them. The amendment was therefore written before analysis. E6c: frontier beats bounded-hier in 34/84 cells ($p = 0.10$) on regime-O and loses on bytes (28/84 cells ($p = 3.0 \times 10^{-3}$)); masking is dominated by bounded-hier in 0/84 cells ($p = 1.0 \times 10^{-25}$). E6a: the break-even price at which frontier becomes cheaper than bounded-hier is 24,444 B pooled (2,148 B independent; never global—bounded-hier wins at every price there). E6d (exploratory runs): at the long horizon the picture splits by family (Table III). E6b is the mechanism-level differentiator: bounded-hier's window leaves 69.0% of due obligations and 35.9% of footprint reads outside its retention, versus frontier's structural 0%—its correctness and cost both ride on the retrieval layer being free and reliable, which the engine grants and a deployment may not. The deployment recommendation (Section IX-A) is updated accordingly: at prices below the break-even, a deployment with reliable retrieval would prefer the bounded window in the simulator; the conversion heuristic remains the boundary of what the model certifies as safe without it.

d) E3: a real trace, replayed.: This repository's own git history, pinned at the pre-analysis commit `2c928d1` (48 non-merge commits; registry of 2,077 file paths), is replayed through the same engine; a replay of the current repository head will differ because the trace grows with the project. E3a (evidence layouts, real change sets only): the trace's train-half density is 0.042, the density rule selects the flat ledger, and the oracle agrees—matched (the tree would have cost 1.71×). The rule's first nonzero-density real test passes. E3b (retirement, hybrid: real invalidation sets with the frozen synthetic obligation/footprint overlay, disclosed as such): frontier cuts operations 23.0% (18.9%–25.9% across seeds) and regime-O 19.6% with 7/7 seeds, both arms—above the 20% operations gate pooled, with the pooled regime-O cut at 19.6% marginally under it. Reported exactly as measured: the real trace's burstier change sets compress frontier's advantage relative to the frozen families, and the live cohort remains the only arbiter of whether that survives real retrieval.

The exploratory results do not change the paper's confirmatory claims; they change its deployment advice and its honesty about the bounded-recency alternative. The live cohort's arms (Section IX-A0b) are unchanged: masking, summary/reset, and frontier remain the arms the frozen closures separate, and bounded-hier enters the live protocol only as a fourth arm if its retrieval-exposure profile can be instrumented there.

F. What the mechanism study does and does not establish

The conversion heuristic and the regime-conditional payoff are structural properties of this simulator: predictions about policies in a deterministic engine, not measurements of agents. The audit checks a policy's capsule against stream ground truth in that engine; a live agent's summaries, retrievals, and obligations are not instrumented by it. The 4,096 B round-trip constant is a frozen accounting choice, not a measured latency; the sensitivity range brackets it but does not replace a live measurement, and every break-even threshold in this paper is a property of that frozen accounting rather than validated deployment guidance. Byte numbers are the frozen accounting of a frozen model of information flow; no hosted model was called, no token or billing claim is made, and the BPE and physical-WAN gaps of Section VIII remain open. The live cohort of Section IX-A0b pre-registers measured retrieval, context, acceptance, and end-to-end outcomes under matched budgets, with operations as its primary cost

TABLE V
PRE-SPECIFIED EXPLORATORY: BOUNDED-HIER AT THE LONG HORIZON ($n = 1{,}024$, 160 EPISODES, SEVEN SEEDS, S = fixed; E6D), BESIDE THE FROZEN FRONTIER ROWS OF TABLE III. BOUNDED-HIER WINS INDEPENDENT AND GLOBAL; FRONTIER WINS CLUSTERED. EXPLORATORY, NOT CONFIRMATORY.

Family	Policy	Ops/run	Regime-O B/run	Regime-B B/run
clustered	bounded-hier	289	3,355,230	2,170,316
independent	bounded-hier	9,826	44,428,129	4,182,003
global	bounded-hier	0	2,097,520	2,097,520

measure precisely because that is where the mechanism result predicts the payoff.

G. The simulator's contract, stated precisely

Because every number above is a simulator output, the simulator's measurement boundary is itself part of the result and is stated here rather than left implicit. The engine is a deterministic function from a seeded episode stream to per-run outcomes; nothing in it calls a model, reads a clock, or observes a network.

a) Stream.: Each episode presents: an invalidation set (gates whose bindings changed), arriving facts (stable identity, verified bit, payload bytes, coupling edges, obligation bit), required reads, and a dependency footprint of D=12 interfaces. Gates persist with probability 0.6 per episode (redrawn on family change); obligations open with a six-episode resolution window; payloads are log-normal with median 128 B. Three families fix the invalidation structure (clustered, independent, global) and are the only source of distributional variation.

b) Policies.: A policy sees the stream through a fixed per-call budget and answers required reads from its own state: submitted context, on-demand fetches, or a lossy summary. The seven frozen arms and their framing constants (fact 32 B, capsule 160 B, binding 16 B, pending 48 B, invariants 64 B, summary 96 B, board row 48 B, recency window 64 gates, reset every 4 episodes) are frozen in the protocol and its engine notes; the audit compares a policy's capsule against the stream's ground truth (retained-generation equality, pending-set equality in both directions, and defective-service counts), not against any external oracle.

c) Acceptance.: A run is accepted iff no required read is served stale and no obligation is lost. Staleness is defined exactly: a read is stale when the generation the policy answers from is older than the stream's current generation for that gate. Loss is exact: an obligation whose gate is not validly read at its due episode is lost. There is no partial credit and no model-mediated judgment anywhere in the acceptance contract.

d) What this boundary excludes.: The engine grants reliable retrieval (every fetch succeeds and returns a current value), exact summaries that drop only what crosses a reset boundary, and payload distributions chosen ex ante. It does not model attention degradation, prompt distraction, formatting drift, provider latency, rate limiting, or billing. Those are the live layer's phenomena; the live cohort (Section IX-A0b) is the pre-registered instrument that measures them, and no claim in this paper extends past the boundary stated here.

V. Sufficiency of the Conversion Heuristic Within the Model

The frozen cohort shows the conversion heuristic is supported at mechanism level: every measured policy that retires before converting fails the acceptance contract. This section proves the complementary direction—*sufficiency*—for the frontier policy under the engine's semantics. What is thereby established is a *model-internal* statement: among the frozen arms of this simulator, the converting policies are exactly the ones that pass, and that statement should not be read as a necessary-and-sufficient retirement rule for agents. The failure direction is a measured property of the seven frozen policies—a lossy summary is the only measured way to fail, but the arm list is not exhaustive over all conceivable policies; sufficiency is a theorem about the engine's semantics. The proposition is scoped to the deterministic engine and to streams with reliable retrieval (every fetch succeeds and returns a current value); the unreliable-retrieval boundary is exactly where the guarantees stop, and Section IV-D measures what bounded-hier's exposure looks like there.

Proposition 1 (Frontier sufficiency)**.** *Fix a finite episode stream $E_0, \ldots, E_{T-1}$ satisfying the frozen stream invariants: (i) every fact's payload is observed before or at its first required read or at its due obligation's resolution episode; (ii) every obligation opened at episode o has its gate in the required reads of episode* $\min(o + W, T - 1)$*, where W is the obligation window; (iii) every required read of episode e is either a current fact of e or fetchable. Then the frontier policy incurs no staleness defects, no obligation losses, and resolves every obligation opened at o with $o + W \leq T - 1$.*

Proof. Staleness. The frontier policy never stores a believed value: a gate enters `known` only through a fresh fact or a successful fetch, both of which set its generation to the current episode, and eviction only removes entries. A required read is answered from the current facts, from a retained entry, or by a fetch (invariant (iii)); the first two are current by construction—retained entries are updated by every observation of their gate—and the third is current by the reliable-retrieval assumption. Hence no read is served stale.

Obligations. An obligation opened at o is inserted into the pending set at o and is exempt from eviction (the frontier retains all pending gates). By invariant (ii) its gate is a required read at $r = \min(o+W, T-1)$. At r, the gate is a current fact, or is retained (the pending exemption keeps it in `known` from opening), or is fetched; in all cases the resolution condition of the engine (the gate was read validly at its due episode) is met, so the obligation resolves. No pending obligation is ever dropped by the frontier policy, so no loss is recorded. For $o+W \le T-1$, $r = o+W \le T-1$, so the resolution happens within the horizon. □

Remark 2. The two directions use disjoint evidence. The failure direction is measured: every lossy-conversion policy in the frozen arm list fails on the frozen grid (Section IV-D); because the arm list is finite and pre-registered rather than exhaustive, this is evidence that lossy conversion is unsafe in this simulator, not a proof that it is the only way to fail. Sufficiency is proved under invariants the frozen stream generator guarantees by construction, which is also why the 84/84 acceptance of frontier runs is a *check* of the implementation against the proposition rather than independent evidence for it. What the model does not prove is the reliable-retrieval assumption itself: in a deployment where re-fetches can fail or return stale state, the conversion heuristic's safety reduces to the retrieval layer's guarantees, and bounded-hier's measured exposure (69.0% of due obligations and 35.9% of footprint reads outside its window) quantifies how much a bounded-recency design delegates to that layer.

a) Retrieval-failure sensitivity.: The reliable-retrieval boundary admits a closed-form sensitivity analysis. The following proposition is a declarative computation over the already-measured operation counts, not a new experiment: the engine's fetches never fail, so no data exists at $p > 0$ and none is simulated here. It quantifies exactly where Proposition 1's guarantee stops, and why the conversion heuristic, not a looser policy, is what survives there.

Proposition 3 (Degradation under unreliable retrieval)**.** *Fix the stream invariants of Proposition 1, replacing invariant (iii) by: each fetch succeeds independently with probability* $1-p$ *for* $p \in [0,1)$*, and a failed fetch leaves its required read unvalidated—recorded as a staleness defect, never zero-imputed. Let* $R(\mathcal{P})$ *denote the number of fetch-dependent required reads of policy* $\mathcal{P}$ *on the stream. Then:*

(i) `full-replay` *has* $R = 0$ *and incurs no retrieval-induced defects at any* p*;*

(ii) $\mathcal{P}$*'s expected retrieval-induced defects equal* $p\,R(\mathcal{P})$*, and its per-run defect-free probability is* $(1-p)^{R(\mathcal{P})}$*;*

(iii) if $R(\text{frontier}) < R(\text{masking})$*, frontier's expected defects are strictly below masking's at every* $p > 0$*, and the ratio is* p*-independent: the measured operations gap carries over unchanged as a defect-rate gap;*

(iv) `summary-reset`*'s defect count is strictly positive at* $p = 0$ *(109,761 stale reads and 385 lost obligations occurred with retrieval fully reliable), so no retrieval layer, however perfect, makes it safe.*

Proof. (i) `full-replay` resubmits all history and issues no fetch. (ii) Each fetch-dependent read fails independently with probability p; the count of failed reads is binomial with mean $p\,R(\mathcal{P})$, and a run is defect-free exactly when all $R(\mathcal{P})$ fetches succeed. (iii) Immediate from (ii) by linearity: $p\,R(\text{frontier}) < p\,R(\text{masking})$ for $p > 0$, and the ratio $R(\text{frontier})/R(\text{masking})$ does not depend on p. (iv) These defects arise from cross-boundary information drops inside the summary itself, not from fetches. □ □

Three quantitative consequences use the long-horizon operation counts of Table III. First, the degradation point against full-replay: frontier's expected excess defects reach one per accepted run at $p \approx 1/284 \approx 3.5 \times 10^{-3}$ in the clustered family and $p \approx 1/8{,}943 \approx 1.1 \times 10^{-4}$ in the independent family; the global family has $R = 0$ and is structurally immune at every p. More generally, a deployment tolerating at most ϵ expected defects per run requires $p < \epsilon / R(\text{frontier})$. Second, the ordering against masking is preserved: frontier's pooled fetch volume is 44.0% smaller, so its expected retrieval-induced defects are 56% of masking's at every failure rate—the conversion heuristic converts an operations advantage into a correctness advantage precisely where retrieval is unreliable, and the break-even price of Proposition 4 is not needed for this direction. Third, bounded-hier enters the same formula with R equal to its out-of-window read volume, which the cohort measured as exposure fractions (69.0% of due obligations, 35.9% of footprint reads) rather than operation counts; its safety delegates to the retrieval layer in exactly the sense of item (ii), with an exposure larger than frontier's structural zero for pending obligations.

The proposition also marks the boundary of what is claimed. A real retrieval layer fails with correlation, staleness, and byzantine responses, not independent Bernoulli trials; retries turn p into p^r only under independent attempts. Neither refinement is measured here. What the model does establish is structural: the conversion heuristic makes the failure surface *countable*—R is a measurable property of the typed capsule—where a summary's failure surface is silent, and item (iv) shows the difference is not a matter of retrieval quality at all.

Proposition 4 (Monotone break-even rule)**.** *Let* $B_f, B_m > 0$ *and* $o_f, o_m \ge 0$ *be per-accepted-run regime-B means and per-accepted-run operation means of two policies, with* $o_f < o_m$*. Define* $R(c) = (B_f + c\,o_f)/(B_m + c\,o_m)$ *for* $c \ge 0$*. Then* R *is monotone on* $c \ge 0$*: decreasing iff* $B_f/B_m > o_f/o_m$*, increasing iff* $B_f/B_m < o_f/o_m$*, constant iff equal; its endpoints are* $R(0) = B_f/B_m$ *and* $\lim_{c\to\infty} R(c) = o_f/o_m$*. For any* $\rho \in (o_f/o_m,\ B_f/B_m)$ *(nonempty exactly when* R *is decreasing),*

$$R(c) \le \rho \iff c \ge c_\rho, \qquad c_\rho = \frac{\rho B_m - B_f}{o_f - \rho\, o_m} > 0. \tag{1}$$

If $\rho \ge B_f/B_m$ *the gate holds at* $c = 0$*; if* $\rho \le o_f/o_m$ *it holds at no* c*.*

Proof. $R'(c)$ has the sign of $o_f B_m - o_m B_f = B_m B_m\,(o_f/o_m - B_f/B_m)$, which is constant in c, giving monotonicity with the stated directions and endpoints. Suppose $o_f/o_m < \rho < B_f/B_m$. Solving $B_f + c\,o_f \le \rho(B_m + c\,o_m)$ for c gives $c\,(o_f - \rho o_m) \le \rho B_m - B_f$. Here $\rho B_m - B_f > 0$ since $\rho > B_f/B_m$, and $o_f - \rho o_m < 0$ since $\rho > o_f/o_m$; dividing by the negative coefficient reverses the inequality, so the gate is equivalent to $c \ge c_\rho$ with $c_\rho = (\rho B_m - B_f)/(o_f - \rho o_m) > 0$. If $\rho \ge B_f/B_m = R(0)$, monotone decrease gives $R(c) \le R(0) \le \rho$ at every $c \ge 0$; if $\rho \le o_f/o_m = \lim_{c\to\infty} R(c)$, then $R(c) > \rho$ at every finite c. The case $o_m = 0$ forces $o_f = 0$, so R is constant at B_f/B_m and only the first endpoint rule applies. □

Remark 5 (Scope of the break-even formula). Proposition 4 converts the frozen sensitivity sweep into a conditional threshold formula. The formula is a property of the frozen accounting: $c_{0.8} = (0.8B_m - B_f)/(o_f - 0.8\,o_m)$ is the price at which the simulator's 20% gate flips, given four means measured in the simulator. On the frozen cohort the byte direction alone does not clear the gate: frontier's byte advantage is 8.0% pooled, below both the 15% and 20% gates, so $c_{0.8} \approx 1{,}364$ B pooled, $c_{0.85} \approx 658$ B pooled, and $c_{0.8}$ is never met in the global family, where both policies are fetch-free and the ratio is constant at 0.924 (Section IV-E). These numbers are simulator thresholds under the frozen accounting, not validated deployment guidance: a real deployment's byte costs, operation mix, and round-trip price are not the simulator's, and the live cohort (Section IX-A0b) is the instrument that would measure them.

VI. Verification Substrate and Serialization

The retirement study of Section IV asks when history can be discarded; this condensed section defines the verification substrate that makes the discarded history reconstructable and auditable—the stable identities, generations, and evidence layouts a typed capsule refers to. The conversion heuristic of Section V consumes this substrate; it does not depend on any particular layout optimizer. These component results carry their v5.1 scope labels unchanged; they are no longer the paper's center, and the new ablation (Table IV) now bounds when their layout choice matters at all. Full derivations that formerly occupied several sections here are consolidated; the exact-DP formulation and its proof now live in Appendix F, and the block-backbone ablation remains in Appendix D.

A. Model and trust boundary

a) Semantic registry, execution, and display.: Let $\mathcal{R} = (g_0, \dots, g_{n-1})$ be a nonempty approved registry of unique check IDs. The candidate identity is

$$e = (\text{epoch},\ \text{snapshot},\ \text{checker},\ \text{environment},\ \text{registry}). \tag{2}$$

The snapshot covers source and protected acceptance inputs. The checker and environment are explicitly bound; an epoch increases even after an ABA return to identical source bytes. Only a trusted runner submits final PASS, FAIL, or UNKNOWN records. Missing, stale, skipped, incompatible, or timed-out outcomes are not passes.

The model splits its state into two disjoint layers. *Immutable check identities* are the registry $\mathcal{R}$, the candidate identity e, and the acceptance predicate each check declares; they are fixed at publication and are never rewritten by execution, layout choice, or learning. *Mutable execution state* is everything a run may change: the result tokens, their generation counters, epoch-local timestamps, and the display positions. An execution permutation σ_e determines when checks run. A layout $L = (\pi, T)$ independently maps tree positions to semantic registry indices. For a node covering positional interval $[i, j]$, its semantic coverage is

$$G_L(i, j) = \{g_{\pi(i)}, \dots, g_{\pi(j)}\}. \tag{3}$$

The stored layout must contain an exact permutation, and every internal node's children must partition its positional interval without gaps or overlaps. The task graph is outside this mathematical tree; one worker can discharge many checks, and one integration check can cover several workers.

The acceptance predicate is independent of both permutations:

$$A(e) = \mathbf{1}\{\text{every } g_i \in \mathcal{R} \text{ has trusted, matching, current PASS evidence for } e\}. \tag{4}$$

An observed fail permits rejection before all checks finish. It does not assert coverage of the unexecuted remainder. Acceptance is relative to the approved predicates, not proof of all program behavior.

b) Packet and lifecycle costs.: Each internal node of arity d emits a fixed-width packet

$$c(d) = H + Sd, \qquad 2 \le d \le b = \lfloor (C - H)/S \rfloor. \tag{5}$$

Here $H = 512$, $S = 320$, and $C = 3072$ bytes, so $b = 8$. A mandatory field that exceeds its slot is rejected, not silently truncated. The cap applies to one certificate packet, not to all project memory, an entire registry manifest, or every LLM prompt. A manifest is a referenced control-plane artifact; its measured size is charged on migration.

Let $W_{e,1}, \dots, W_{e,q_e}$ be the semantic IDs whose final records become available between successive refreshes. For a strict new candidate epoch, the observed cost is

$$\ell_e(L) = W(L) + \sum_{a=1}^{q_e} \sum_{v \in \mathrm{int}(T)} c(d_v)\,\mathbf{1}\{W_{e,a} \cap G_L(v) \neq \varnothing\},$$
$$W(L) = \sum_v c(d_v). \tag{6}$$

The first term constructs the full initially unknown tree. A successful strict epoch runs the whole registry. We separately study persistent-state refresh traces that omit $W(L)$ at each tick, as in a cache with correctly specified incremental dependencies; each topology installation is still cold and explicitly priced. These traces evaluate the

controller's cost model; they are not the strict candidate-gate timing experiment, and they do not demonstrate safe dependency inference from arbitrary code.

c) Why more than invalidation or pairwise similarity is needed.: A classical result shows that marginal check activation cannot identify the best aggregation tree under joint invalidation (Appendix E), and even the full initial invalidation distribution cannot identify optimal progressive verification. One further construction bounds what pairwise statistics can see:

Proposition 6 (Pairwise affinity is insufficient for full-wave scoring)**.** *There exist two distributions with identical singleton and pairwise co-activity probabilities, hence identical Jaccard affinities, but different activation probabilities for the same three-check subtree.*

Proof. On IDs $\{0,1,2\}$, let P be uniform on $\varnothing, \{0,1\}, \{0,2\}, \{1,2\}$ and Q uniform on $\{0\}, \{1\}, \{2\}, \{0,1,2\}$. Every singleton has probability $1/2$, every pair has joint probability $1/4$, and its Jaccard affinity is $1/3$. Yet the probability that the triple is hit is $3/4$ under P and 1 under Q. Thus a packet emitted on that interval has different expected cost. □ □

The implication for the compiler is specific: a pairwise clustering objective cannot substitute for scoring recorded completion hyperedges.

B. Learning layouts without changing the contract

a) Practical catalogue and full-wave selection.: Training waves define counts N_i, N_{ij} and affinity

$$a_{ij} = \frac{N_{ij}}{N_i + N_j - N_{ij}}, \tag{7}$$

with zero for an unseen pair. Affinity orders semantically related checks canonically. All semantic IDs, including disconnected and unseen checks, remain included exactly once.

Given the learned order, group consecutive leaves into blocks of at most eight, promote a singleton unchanged, and repeat upward. This bottom-up compact balanced construction does not call a tree optimizer. The catalogue retains the incumbent first, includes original-order fixed8 when distinct, and proposes learned-order fixed8. Identical layouts are deduplicated. A separate validation cohort scores complete completion waves, including initial construction; ties retain the incumbent. The API returns an advisory choice, not deployment authorization. The registry and packet cap cannot change during selection.

For any proposal π, training episodes supply interval weights

$$r_\pi(i,j) = 1 + \frac{1}{m}\sum_{e=1}^{m}\sum_{a}\mathbf{1}\{W_{e,a} \cap G_L(i,j) \neq \varnothing\}. \tag{8}$$

Pairwise affinity proposes an order; it does not replace the full-wave cost. The practical catalogue contains at most three candidates and uses no DP. Tree construction is $O(n)$ after ordering. The reference implementation retains spectral ordering and exact fitting for research comparisons; neither is invoked by the practical default. The exact conditional tree synthesis—its dynamic programming recurrence, exactness theorem, and proof—is retained unchanged in Appendix F for reference; the practical default never calls it.

Proposition 7 (Validation after learned ordering)**.** *Condition on a training set that fixes K complete layouts. For m independent identically distributed validation epochs with each layout cost in $[0,B]$, let $\widehat{L}$ minimize empirical validation cost. With probability at least $1-\delta$,*

$$J(\widehat{L}) \le \min_{L\in\mathcal{C}} J(L) + 2B\sqrt{\frac{\log(2K/\delta)}{2m}}. \tag{9}$$

Proof. Conditionally, the catalogue is fixed. Hoeffding's inequality and a union bound over K layouts bound every empirical mean error by the displayed radius without its factor two. Inserting empirical costs on either side of the empirical minimizer yields the result. □ □

The guarantee is only relative to the fixed catalogue. It neither justifies choosing among learned permutations on their training errors alone nor covers distribution shift. We do not report this potentially loose bound as an empirical confidence interval; reported intervals use independent experimental seeds.

b) Compiler identity and scope.: The practical default has no hidden microtree optimization. Exact DP is a reference and optional offline candidate (Appendix F). The block-backbone proposition and implementation remain in Appendix D solely as an ablation; its restricted-class exactness does not override the observed simpler-baseline advantage. At 1,024 checks, order learning still takes seconds in our measurement, so layouts should be cached and compiled away from the agent's per-action path. Validation of padded costs does not optimize compressed archives or model tokenization; adoption must measure the actual delivery representation.

C. Practical online selection: price the window, not the promise

Freeze a compiled catalogue $\mathcal{C} = \{L_1, \ldots, L_K\}$. At event t, choose a layout before observing the current completion wave; afterwards score that same wave under every catalogue member. This is full-information feedback only because the check schedule is held fixed. It is not counterfactual feedback for arbitrary agent behavior.

Algorithm 1 Deployable evidence control; inference and checking remain separate

for each build request **do**
  Build context only from the semantic registry
  Check out the frozen, identified source snapshot
  If required, select layout offline via the frozen rule
  if layout changes **then**
    Revoke tickets, increment generation, remap stable IDs, rebuild and price the installation
  end if
  Receive trusted completion records for the fixed check schedule
  Reject stale, conflicting or malformed records; aggregate bounded packets
  Publish only the complete current contract result; otherwise retain FAIL/UNKNOWN
  Expose a root-status card; retrieve diagnostic packets only when needed
  Score the observed event and append its cost vector to the bounded window
end for

A different layout is installed by serializing its canonical manifest and reconstructing its complete certificate tree:

$$M_{ij} = \begin{cases} 0, & L_i = L_j, \\ |\mathrm{manifest}(L_j)| + W(L_j), & L_i \neq L_j, \end{cases}$$

$$L_T = \sum_t \ell_t(A_t) + \sum_{t=2}^{T} M_{A_{t-1},A_t}. \tag{10}$$

The initial installation is separate. These byte prices match the evaluated uncompressed migration protocol. A deployment using compression must measure its own priced wire representation; the DP objective is not automatically an optimum for a compressed log.

a) The primary practical rule.: Let $\bar{\ell}_{t,i}$ be the mean cost on the last $\min(w, t-1)$ observed events, and $j_t = \arg\min_i \bar{\ell}_{t,i}$ with deterministic ties. Starting from the calibrated incumbent a, use the frozen rule

$$A_t = \begin{cases} j_t, & w(\bar{\ell}_{t,a} - \bar{\ell}_{t,j_t}) > M_{a,j_t}, \\ a, & \text{otherwise}, \end{cases} \qquad w = 64. \tag{11}$$

With no history, keep a. A tie in estimated savings does not trigger migration. During warm-up, the forecast horizon remains w even if fewer events were observed, matching the original baseline. This is an empirical break-even forecast, not proof that past savings repay a future switch. It can lose under abrupt reversal; preserving the frozen layout is always a legitimate alternative.

The runtime API enforces `choose()` before `observe()`, bounds history to 64 vectors, rejects nonfinite costs, and checkpoints only at event boundaries. Replaying all 80 archived streams, with periodic checkpoint/resume, must reproduce the original baseline's full action and byte-cost paths exactly. The exercise validates implementation equivalence, not a new performance dataset.

b) Why fixed-share is no longer primary.: On the archived stationary, long-shift and rapid-shift streams, the coupled fixed-share policy is more expensive than the priced window. Its standard tracking analysis therefore moves to Appendix C, with all negative results retained. We do not manufacture an adversarial workload merely to make that policy win. Neither the appendix bound nor the empirical window result establishes general superiority over other switching-cost policies.

D. Stable evidence through permutation and migration

A result token carries (epoch, g_i, b_i), where b_i binds the semantic ID to the registry, source snapshot, checker, and environment. It does not carry a display position as its identity. A layout-specific kernel maps stable IDs to current positions. During migration it rebuilds from canonical trusted records; it never transplants an ordinal cache entry into a different permutation.

Publication tickets additionally carry a monotone layout generation and layout hash. A layout switch revokes the issued ticket, advances the generation, and forces a full recheck of the affected subtree. The registry, candidate identity, and acceptance predicates never change through a layout-only change, but a new candidate epoch fences it. Duplicate identical evidence is idempotent. Conflicting final records in one epoch revoke authorization: a recorded fail remains a fail; otherwise the obligation becomes unknown. Resolution requires a new epoch, not a more persuasive agent summary.

Theorem 8 (Layout-independent local publication soundness)**.** *Assume complete approved predicates and dependency identities, a trusted checker executing immutable identified inputs, collision-resistant binding, and atomic local epoch/generation checks. For any sequence of valid layout permutations, any advisory learning outputs, and any delivered evidence order, authorization implies Equation* (4)*. If each checker is sound for its declared predicate, those predicates hold on the authorized candidate.*

Proof. A canonical pass requires a current token for the same semantic ID and immutable identities. Permutation is a bijection, so rebuilding moves each such record to exactly one position without changing its meaning. Exact child partitioning preserves coverage and pass/fail/unknown counts by induction up the tree. The root can be authorized only with $(n, 0, 0)$ and matching current registry, epoch, generation, and layout. Old tickets fail those checks. Conflicts prevent authorization. No choice made by the learner writes trusted outcomes or changes the registry. □ □

This is a conditional local guarantee, not remote attestation or distributed consensus. An incomplete suite, compromised checker, omitted dependency, or nondeterministic external service can invalidate a broader correctness claim. The CLI's file copy is not an OS sandbox. An external merge/deployment service must atomically compare

the verified candidate and integration-head identities at commit; a report file alone is not that transaction.

E. Implementation and cost accounting

The artifact adds `practical.py`, `deployment.py`, `token_budget.py` and `tls_probe.py`. The practical CLI produces a balanced catalogue without calling either exact or block DP. The historical exact and block compilers remain separately callable. The existing stable-ID guard, priced window, codec and Future Code bridge are retained. The strict CLI compares protected input sets and hashes, detecting added tests or configuration files as well as changed and deleted ones. Schema-2 contracts bind the approved checker and environment and refuse silent reuse of legacy contracts. Source manifests are checked before and after snapshotting and verification; mutation of an observed snapshot prevents a pass.

The bridge runs as an ordinary subprocess acceptance gate in the unmodified Future Code v1.2 runtime. The supervisor still owns worker leases, private attempts, bounded contexts, and serialized integration. Non-pass exits cause candidate rejection. Detailed evidence and binary certificate packets live outside candidate write scopes; only a bounded JSON result card returns to the worker. The plug-in accepts a precompiled layout. The online selector and migration guard are separately integrated and stress-tested as a library path; a live, self-retuning hosted deployment was not evaluated.

A different scalar objective could combine these quantities only with declared unit conversions and observable counterfactual costs. This implementation does not optimize unknown model latency or billing. The exact-byte claim also depends on the stated padded format; compression defines a different, potentially nonseparable objective for which this DP is not claimed exact.

Costs are accounted in separate ledgers so byte savings are never converted into unsupported latency or token claims:

VII. Service-Matched Delivery and Admission

A. Do not conflate three consumers

The local aggregator receives bounded packets; a logging collector may receive their complete stream; a supervisor normally receives a status projection and retrieves details selectively. Define $\psi(e, c)$ as canonical status text containing semantic identity, counts and local authorization, but no display positions. For the same trusted semantic report,

$$\psi_{L_1}(e, c) = \psi_{L_2}(e, c). \tag{12}$$

Its byte length and token count under any fixed deterministic tokenizer are therefore identical. This identity is not a new theorem about tokenization. It supplies a negative control: layout savings cannot reduce a layout-independent root input. The card is a projection of a trusted report, not authority to accept arbitrary worker JSON.

Diagnostics preserve full header/card records and page whole packets under a 16,384-byte limit. A packet that cannot fit is rejected, not truncated. For a real supplied tokenizer τ and explicitly reserved framing budget r, the new admission API additionally requires

$$|p|_{\mathrm{UTF8}} \le C_{\mathrm{bytes}}, \qquad |\tau(\mathrm{decodeUTF8}(p))| + r \le C_{\mathrm{tokens}}. \tag{13}$$

The implementation retokenizes each full proposed page, including its prefix. Token counts are not assumed additive across concatenations. Greedy packing guarantees the tested limits, not a minimum number of pages. Toy counters in API tests are not BPE measurements. The optional measurement command requires actual tiktoken software and vocabulary [20], fingerprints the encoding, and records per-page content hashes. Missing encodings produce UNKNOWN and null counts; there is no byte-ratio fallback. Provider framing and billed usage require a separately measured adapter, not a guessed constant.

B. Encoding and the simplest sufficient service

Canonical unpadded JSON, independently compressed packets, and whole-epoch DEFLATE preserve their input records with different operational boundaries. Whole-epoch compression requires the completed archive, whereas per-packet compression permits incremental export. The padded objective is not an exact optimizer of either compressed bytes or tokens. Practical selection is advisory until its gain is measured with the deployed codec.

For a trusted consumer needing only final status, a root endpoint or flat ledger can be much cheaper than the intermediate stream. This changes the diagnostic service, not necessarily the underlying check set. Comparing that smaller service with an exhaustive stream cannot demonstrate that a tree optimizer is efficient or necessary. The strict checker, data representation, and delivery demand must each be named explicitly.

C. Adoption cost is not marginal layout cost

Consider q copies of the same workload and delivery service through one nonoverlapped bottleneck. Let B_0, B_1 be codec-matched application bytes per copy, $R > 0$ effective application goodput, $E \ge 0$ additional encoding/decoding work per copy, and $C \ge 0$ consistently amortized one-time incremental work. Common setup cancels only when actually shared. The conditional net benefit is

$$d = (B_0 - B_1)/R - E, \qquad \Delta(q) = qd - C. \tag{14}$$

A strictly profitable integer copy count exists only when $d > 0$, and then

$$q_{\min} = \lfloor C/d \rfloor + 1. \tag{15}$$

This is accounting, not a new networking or regret theorem. A zero gain retains the baseline. The implementation uses decimal arithmetic at equality and rejects mismatched service IDs. Missing C, E or R returns UNKNOWN, not

TABLE VI
Separate ledgers avoid converting byte savings into unsupported latency or token claims.

Quantity	Instrument	Ledger
Serialized certificate bytes	Sum over packets with fixed per-node header and slot widths	Bytes; never tokens
Wall-clock durations	Monotonic clock around whole-run driver and per-stage timers	Seconds; never bytes
Tool calls	Counted supervisor-side	Counts
Check latencies	External driver timeouts are host-configured and enforced per tool call	Reported separately
Migration bytes	Canonical layout manifest plus forced full rebuild	Yes, online traces
Fit and scoring time	Monotonic-clock measurements	Reported separately
Preparation, copying, checking, aggregation	Wrapper timers and outer call duration	Reported separately
Active harness episode	Supervisor startup through completion and shutdown	Reported separately
Prompt bytes and construction time	Captured actual HTTP messages and context builder	Reported; not tokens
Hosted token usage and API billing	No provider inference was run	Unknown, never zero-imputed

zero. An explicit zero is permissible only as a declared assumption or measurement.

When both variants already use the same protected gate, C is extra layout work, not the full strict-wrapper cost. When adopting strict verification from ordinary testing, its additional cost must be justified independently; the services differ and the planner returns `INCOMPARABLE`. Repeating exports amortizes one candidate's preparation, not the preparation of future new candidates. Parallel links, computation overlap, cached diagnostics, queueing and different connection policies require direct critical-path measurement. High RTT alone does not make byte savings larger if both exports pay the same setup and request count.

D. Remote measurement and security boundaries

The historical transfer experiment is receiver-paced loopback, not a WAN. The new opt-in measurement kit uses verified TLS server certificates and hostname checking; upload and nonlocal bind require explicit switches. Frames and decompressed archives are bounded, and acknowledgments check received length and digest. Local tests exercise cold and reused TLS sessions, rejected hostnames, untrusted roots, malformed acknowledgments and truncated records.

These tests validate a protocol path only. They provide no physical-WAN throughput, jitter, packet-loss or retransmission measurement. The supplied protocol records host and path provenance, codec, connection policy and matched payloads, and requests separately observed RTT/goodput and transport diagnostics where available, consistent with the distinctions in TCP throughput testing [22]. Fields that the minimal client cannot observe remain null. There is no receiver-side artificial pacing in a qualifying WAN trial. No public upload, account creation or credential search was performed.

TLS authenticates the configured server, not the checker's truthfulness. Protected reference data, immutable candidate identity and local monotone state remain required for Theorem 8. There is no remote attestation, Byzantine consensus, client authorization service, non-repudiation or OS sandbox. Replacing both evidence and its trusted reference still defeats a digest-only comparison. The receiver is a bounded measurement utility to use behind an operator-controlled firewall, not a production ingestion service.

VIII. Evaluation of the Practical Revision

A. Provenance and frozen questions

The v5 contract was saved before the new runs. It fixes three sizes (128, 512, 1,024), three completion families (clustered, independent, global), and three seeds (55011–55013). Each of the 27 workloads has 128 training epochs, 128 validation epochs and 256 held-out epochs. Random draws are sequential disjoint samples; deterministic global observations naturally coincide. The catalogue is fit only on training data, selected on validation, and reported on held-out data. Cold strict-epoch construction is included in every objective. The nine size/family summaries use seed means and observed ranges, not episode-pooled significance tests.

A separate study re-aggregates twelve archived source candidates under three layouts, producing 36 layout replays. All use the same original-order trusted check records, the same executed sequence and batches of eight. Average-linkage orders were learned on the earlier training cohort and frozen before these candidates. None of these replays is a fresh checker execution, independent code repair, or new agent trial. Historical v3/v4 data remain in the appendices with their original cohort identities.

TABLE VII
New v5 synthetic study. Reduction is selected-layout held-out padded cost relative to the original-order balanced incumbent; positive is better. Fit includes affinity/order learning and catalogue construction, not validation scoring. Three seeds per row; ranges are descriptive.

Checks	Family	Mean (%)	Range (%)	Fit median (s)	Changed
128	cluster	23.43	22.40–24.45	0.018	3/3
128	independent	0.00	0.00–0.00	0.016	0/3
128	global	0.00	0.00–0.00	0.032	0/3
512	cluster	10.75	10.06–11.13	0.523	3/3
512	independent	0.07	0.00–0.20	0.548	1/3
512	global	0.00	0.00–0.00	0.645	0/3
1,024	cluster	5.56	5.16–5.80	2.991	3/3
1,024	independent	0.00	0.00–0.00	3.086	1/3
1,024	global	0.00	0.00–0.00	3.462	0/3

B. Practical default without exact or block DP

On clustered completions, mean held-out reductions are 23.43%, 10.75% and 5.56% as registry size increases (Table VII). Global updates give no reduction; independent updates give zero except for one of the three 512-check seeds, whose small gain produces a 0.07% mean. These controls do not support universal value from ordering. The selected method includes no DP, and a regression test makes any call to the exact fitting helper fail on this path.

The full catalogue fit median at 1,024 checks remains about 3 seconds; order learning is not free. The fixed construction can reuse a previously approved order in linear work, but no linear-time claim is made for the full learned pipeline. The smaller percentage at larger registries is consistent with substantial layout-independent cold construction in this benchmark; it should not be interpreted as a learned scaling law.

An initial execution exceeded its 120-second tool limit after saving 26 complete records. No process remained; explicit resume skipped those exact records and completed the remaining frozen seed without changing data or selection. All 27 observations and the interruption record are retained.

C. Source-evidence replays: simplicity is not byte optimality

Learned balanced layouts reduce whole-epoch compressed bytes by 41.84% for NetworkX, 4.36% for Toolz and 18.53% for Future Code (Table VIII). These are not the larger v4 exact-layout savings relabeled as results of the new default. The balanced representation remains larger than learned exact in every project here. Exact fitting may therefore be appropriate when amortized delivery justifies its additional compilation; the practical default trades that potential gain for simpler fitting.

Across the twelve cases, batch-compressed application bytes are 416,046 for fixed8, 299,081 for learned balanced and 278,831 for the re-serialized learned exact layout. Semantic counts, verdicts and local authorization agree across all 36 replays. Their provenance binds the archived check records by digest. Serialization includes newly bound layout/run metadata; numerical bytes need not exactly reproduce a prior archive's compressibility. The independent audit checks actual emitted bytes rather than substituting earlier totals.

Root-status text is identical in all twelve triples and occupies 593–594 UTF-8 bytes. Exhaustive diagnostic pages total 148 for fixed8, 95 for learned balanced and 87 for learned exact. The new default therefore uses 35.81% fewer pages than fixed8, not v4's 41.22% exact-layout reduction. The largest emitted page is 16,364 bytes under a 16,384-byte limit. Reading every page is a specified exhaustive diagnostic service, not observed LLM behavior. There is no measured token reduction; root token equality follows from identical text for a fixed tokenizer, but no numerical BPE counts are asserted.

D. Codec-matched deployment scenarios, not measured speedups

The measured fixed-to-balanced compressed byte difference is 116,965 per twelve-candidate cohort. Equation (14) gives 1.784744 seconds of gross delivery saving per cohort copy at an assumed 64 KiB/s goodput, with $E = 0$. This is arithmetic from replayed byte sizes, not the older 2.21-second measured loopback saving of a different layout.

The 5-second column (Table IX) is a deliberately declared sensitivity scenario, not an estimate transferred from the historical harness. At 64 KiB/s and one export per candidate, the assumed 60 seconds of cohort overhead overwhelms the approximately 1.785-second delivery saving. Repeated delivery might change the equation, but the application must actually require it. With new candidates on each repetition, preparation recurs and cannot be amortized as the same C. All 27 scenarios (three codecs, three rates, three overhead assumptions) are available, including negative outcomes. The planner rejects mismatched services and missing measurements rather than inventing a benefit.

E. Historical controls remain controls

The earlier 288 loopback transfers belong to v4, use application pacing rather than real WAN conditions, and are

TABLE VIII
NEW SERIALIZATION OF TWELVE ARCHIVED SOURCE CANDIDATES, FOUR PER PROJECT. THE FIRST THREE COLUMNS COMPARE LEARNED-ORDER BALANCED8 WITH ORIGINAL-ORDER FIXED8. THE LAST COLUMN COMPARES LEARNED BALANCED WITH LEARNED EXACT AFTER EPOCH COMPRESSION: POSITIVE IS A PENALTY. ALL INCLUDE THE CODEC'S APPLICATION FRAMING.

Project	Padded (%)	Canonical (%)	Epoch (%)	vs. exact penalty (%)
networkx	48.63	48.74	41.84	11.68
toolz	8.78	8.70	4.36	1.52
future	29.12	29.23	18.53	6.67

TABLE IX
CONDITIONAL MINIMUM PROFITABLE COPIES OF THE SAME TWELVE-CANDIDATE COHORT WITH WHOLE-EPOCH COMPRESSION. $E = 0$ IS AN ASSUMPTION. THE COLUMNS ASSUME EXTRA COST PER CANDIDATE, PAID ONCE; COHORT COSTS ARE 0.6, 6 AND 60 SECONDS. THESE ARE NOT MEASURED STRICT-WRAPPER OVERHEADS.

Assumed goodput	0.05 s/candidate	0.5 s/candidate	5 s/candidate
8 KiB/s	1	1	5
64 KiB/s	1	4	34
1 MiB/s	6	54	538

not rerun or relabeled as v5 trials. For twelve compressed fixed/exact exports at 64 KiB/s, their measured totals were 6.624 and 4.413 seconds. The larger uncompressed totals were 33.499 and 18.954 seconds. Compression of fixed8 was more impactful than selecting an uncompressed learned layout. None of those durations is an IP/TLS throughput measurement or predicts a loss-prone path.

The original 39.69/39.77-second checker totals and 12.98/18.11/18.12-second median direct/fixed/learned harness durations remain negative controls. They came from different source/harness fixtures, so this paper does not subtract their times across cohorts. The historical window-controller and block-compiler evidence is retained unchanged. Block DP's 1.54–5.34% penalty to same-order balanced8 is the reason it is now an ablation, not a contribution advertised as deployment acceleration.

F. Unmeasured endpoints and tested interfaces

Both requested BPE encodings, `cl100k_base` and `o200k_base`, remain UNKNOWN: the software was absent, package/vocabulary fetches failed, and a connector-retrieved build artifact was expired. No substitute tokenizer, model alias or byte division was used. The measurement tool records exact content-only counts when a real encoding is available; its unit tests use explicitly labeled toy counters only to validate admission logic.

No two authorized remote measurement hosts or designated live model endpoint were available. Physical-WAN and hosted-LLM trial counts are zero. The new TLS utility's local compatibility tests check transport integrity, certificate rejection and bounded framing; they do not measure WAN jitter/loss or close the agent-evaluation gap. A reproducible deployment protocol is supplied, rather than a claimed remote experiment.

G. Software and independent audit

The prototype suite now spans both protocol generations of the retirement study: the v1 engine's 10 pinned invariants (determinism, summary/reset failure, full-replay coverage, flat steel-man accounting, closure outcomes) and the v2 engine's 12 (factorial coverage, scheduler byte-neutrality, phase-sum identity, audit cleanliness, closure directions, long-horizon persistence, bit-identical fast layouts). The v2 suite caught a real defect during development—the audit's ground-truth pending set deduplicated obligation openings by first rather than latest opening—which is disclosed in Section IV-D; the frozen cohort was regenerated before any v2 manuscript text was written. The earlier serialization prototype passes 329 tests, including 76 new cases; the existing version-equality assertion is updated to require 5.0.0 consistently in runtime and package metadata. A duplicate test-module basename and the initially inconsistent version were corrected without weakening behavior checks; failure logs are retained. New tests cover no-DP compilation, service mismatch, missing costs, exact break-even, whole-text token admission, TLS trust failures and deliberate data tampering.

A separate auditor imports no producer optimizer, selector or codec. It reconstructs full-initialization and wave costs for all 27 workloads, checks 36 serialized replays against original evidence digests and verdicts, reconstructs all codec byte sizes and diagnostic pages, and independently checks 27 planning scenarios and 131 summary comparisons. These checks validate recorded arithmetic and provenance, not third-party security certification. Release verification separately records extracted-source tests, the unchanged Future Code suite, installed-source hashes and archive integrity.

IX. Deployment Recommendation, Limitations, and the Pre-Registered Live Cohort

A. Recommendations from the simulator, and the protocol-only tier

The recommendations below have two tiers: what this paper's simulator measures, and what remains protocol-only.

a) Mechanism-level (measured in this paper's simulator).: Where a service already requires protected verification evidence and a constrained channel, the serialized-evidence component of Section VI applies as before. The retirement study adds a sharper pattern for context policy: in the simulator, the policy that first converts history into current verified bindings, a typed pending-obligation set, and dependency interfaces passes the acceptance contract at every scale tested, while policies that rely on lossy summaries of obligations, or that revert values to beliefs older than the current working window, measurably fail (Section IV-D). The conversion is the pattern the simulator supports; whether it is worth adopting depends on the cost regime and on the retrieval layer. It buys 44.0% fewer retrieval round trips at equal correctness, and by the closed form of Proposition 4 the simulator's gate flips at a computable round-trip price: given means (B_f, B_m, o_f, o_m) measured in a deployment's own pilot and that deployment's price c, the formula's threshold $c_{0.8}$ states where the frozen 20% gate would flip. In the simulator, $c_{0.8} \approx 1{,}364\,\mathrm{B}$ pooled (frontier's 8.0% byte advantage does not by itself clear the 20% gate; the frozen 4,096 B round-trip price does). These are simulator thresholds under the frozen accounting, not validated deployment guidance: they identify what a deployment must measure—its own byte costs, operation mix, and round-trip price—rather than prescribing adoption. Second, and now stated because the exploratory analyses forced it: a 64-gate bounded-recency window (`bounded-hier`) is cheaper than frontier below a break-even price of 24,444 B pooled (2,148 B independent; never in global) at equal acceptance—but 69.0% of its due obligations and 35.9% of its footprint reads sit outside its retention, so its safety delegates to the retrieval layer. The simulator-conditional reading is therefore a function of two observable quantities, not one: with reliable, cheap retrieval and round-trip price below the break-even, the bounded window is the cheaper arm in the simulator; with expensive or unreliable retrieval, the conversion heuristic is the arm whose model-internal guarantees do not depend on the retrieval layer beyond the fetch itself. Which of these a real deployment should prefer is exactly what the live cohort measures; the simulator brackets the possibilities rather than deciding between them. The real-trace replay (Section IV-E) brackets what this looks like on non-synthetic change patterns. Between grouped certificate layouts and flat delta ledgers, select by update density: the measured-density rule matched the per-cell oracle on every cell of the frozen cohort (0.0% regret); dense updates favor the tree, sparse deltas favor the flat ledger, and the density signal is observable on a training half of any real stream. The demand-driven adaptive variant is not promoted: its pre-registered gate failed in the simulator (−0.0% versus plain frontier), and adoption decisions belong to the live cohort, not to the simulator alone.

b) Live-agent cohort (pre-registered, not run; highest-priority next experiment).: The mechanism study predicts, but cannot confirm, that avoided retrieval operations translate into live savings. Executing this cohort is the single most informative next step the study permits itself: it is the only pre-registered instrument that can move real-world agent relevance from moderate to measured, and every limitation this section records—BPE token counts, provider billing, prompt-semantics effects, and live tool-call latencies—is either an outcome it measures or a risk it prices. The confirmatory cohort is therefore pre-registered with operations as the primary cost measure:

1) **Workload.** Multi-stage changes on a fixed public issue benchmark (SWE-bench-class), screened for dependency coupling; the same frozen independent acceptance contract for all arms; equal total budgets, sandbox, and protected state; every failed branch in the numerator.
2) **Arms.** The arms the simulator separates: observation masking; summary/reset; verified-frontier retirement; and the density-selected evidence layout (flat delta ledger versus grouped tree, chosen per the measured update density of the pilot). The demand-driven adaptive variant is excluded because its promotion gate failed in the simulator; including it would re-litigate a pre-registered decision.
3) **Primary outcomes.** Tool-call round trips per accepted project (the pre-registered live-cost measure, from the mechanism result); cost per accepted project (all provider metering preserved, unknowns never zero-imputed); acceptance rate; deadline success with censored timeouts. Pre-specified noninferiority margin on acceptance: no arm may trail masking by more than 5% absolute.
4) **Closure rules.** *Null closure*: if frontier retirement does not reduce tool-call operations per accepted project by at least 20% versus masking at noninferior acceptance, the live-cohort hypothesis is rejected and retirement is not promoted. *HACT closure*: if the density-selected layout does not beat flat-delta evidence by at least 5% on measured evidence bytes at equal correctness, the tree is not deployed.

Two explicit closure rules prevent indefinite iteration; either rule terminates the program with a written result. This protocol is frozen here, before any live data collection.

The live layer also prices risks the mechanism study cannot express and must not silently assume away. A hosted model does not read idealized state capsules: structured JSON or capsule formats can induce attention degradation, prompt distraction, or formatting drift, so the cohort scores each arm's full delivered prompts, not just their

sizes. Real summarization does not follow a synthetic four-episode boundary, so the summary/reset arm keeps its own provider-side implementation with the acceptance contract as the only fixed point. And live round trips are not a static additive constant: network jitter, rate limiting, and variable generation latency replace the frozen 4,096 B accounting constant with the cohort's measured per-tool-call durations, which is why tool-call counts, not byte totals, remain the pre-registered primary outcome.

c) What remains unmeasured.: No hosted model was called for any number in this paper; BPE tokenization, physical-WAN behavior, and provider billing remain unknown and are never zero-imputed. Two tokenization caveats deserve emphasis because the paper's savings are reported in bytes. First, both requested BPE encodings (`cl100k_base` and `o200k_base`) remain UNKNOWN (Section VIII-F); byte savings therefore do not necessarily convert to proportional BPE token savings, and no byte-to-token division is asserted anywhere. Second, even a measured conversion would be model-specific: tokenization is not a length-preserving map, so the live cohort records actual tokenizer output per arm rather than converting the byte result. The mechanism cohort's cost model is a frozen accounting of a frozen model of information flow; real payloads are not lognormal, and real summaries may retain obligations better or worse than the seeded boundary survival. The round-trip regime's 4,096 B constant is an accounting choice bracketed by a sensitivity range, not a measured latency. The tool-call prediction is the only bridge from the mechanism result to live cost, and it is pre-registered as such rather than assumed.

d) Trust and scope.: Future Code remains a single-host supervisor. The guard and TLS utility are not a distributed consensus protocol or fault-tolerant deployment. A passing declared suite cannot establish absence of arbitrary defects. Authorization requires an uncompromised checker, correct protected reference data, immutable candidate identity, and local monotone state; Theorem 8 stands as before. TLS authenticates the configured server, not the checker's truthfulness. The receiver is a bounded measurement utility behind an operator-controlled firewall, not a production ingestion service.

Appendix A
Audit and Reproduction

A. Exact data generation

The synthetic generator uses NumPy's `default_rng` initialized with a `SeedSequence` whose entries are `[260918, n, seed, stream]`. Stream zero produces training episodes and stream one held-out episodes. Group construction uses `SeedSequence([7001,seed,n])`. Pilot seeds 1000 and 1001 are excluded from all main tables. Scaling sizes other than 128 have their own saved records; the four reused 128-gate seeds are clearly identified in the analysis.

The v5 frozen contract fixes sizes 128, 512 and 1,024; families clustered, independent and global; and seeds 55011–55013, with 128 training, 128 validation and 256 held-out epochs per workload. The executable fixture uses `SeedSequence([83012,seed,stream])`. Interrupted fixture runs append completed snapshot records to a journal and replay those records when resuming; they do not reclassify incomplete work as a completed seed.

The main commands, from the extracted research root, are:

```
python -m pytest -q
python -m experiments.synthetic \
    --family clustered --n 128 --seeds 20
python -m experiments.executable \
    --seeds 6 --out results/executable_fresh
python -m experiments.sensitivity
python -m experiments.aggregate
python -m experiments.validate_results
```

Nothing in these commands contacts an LLM endpoint. The artifact separates raw results from regenerated tables.

B. Software and data identities

Archived runs used Python 3.13.5 on Linux x86_64; exact installed package and platform records accompany emitted reports. Compilers were measured with `OPENBLAS_NUM_THREADS=1`. No accelerator is required. Vendored upstream sources and tests are retained with their notices. Every fresh mutation is checked against its archived base-source hash, replacement span, and resulting source hash. This binds a result to a specific intervention, not a natural historical bug distribution.

Appendix B
Certificate Encoding and Trust-Boundary Details

Each capsule contains `range`, `counts`, `binding`, `evidence`, and `registry`. Hash fields are 64-character lowercase SHA-256 hexadecimal encodings. Canonical JSON uses sorted keys, ASCII escapes, and compact separators. In a packet, the header occupies 512 bytes including its newline; each child card occupies 320 bytes including its newline. Padding makes byte accounting exact. Fixed padding is not asserted to be a token-optimal production encoding; a binary or tokenizer-aware encoding would define a new price function and require re-evaluation.

The trusted verifier API rejects mismatched bindings, invalid verdicts, and malformed trace hashes. It is intentionally not exposed as a tool through which an untrusted LLM can certify its own work. Returned display capsules are compared with canonical trusted records rather than accepted because their self-reported hashes appear valid. This local design demonstrates record integrity and freshness under its trust assumptions; distributed authenticated transport and credential isolation remain host-runtime responsibilities.

The optimizer's $O(bn^3)$ claim concerns planning. The reference kernel prioritizes clarity and scans interval generation stamps; it also hashes tree descriptions. We do not infer an asymptotically optimal runtime-update implementation from the communication bound. Large-scale implementations could use range-maximum indexes and cached topology digests, but those optimizations are outside the evaluated prototype.

Appendix C
Optional Fixed-Share Analysis: Not the Deployment Recommendation

This appendix retains the former tracking theorem and its assumptions. It is an optional comparator, not the practical selector. On the measured streams it loses to the deterministic priced-window rule in three regimes. We have not searched for a favorable adversarial stream after seeing those results, and make no claim that a tracking bound implies an empirical advantage.

Let a declared bound R cover each round's loss spread, $\max_i \ell_t(i) - \min_i \ell_t(i) \le R$. Reject a measurement exceeding it instead of clipping the cost. Define $z_{t,i} = (\ell_t(i) - \min_j \ell_t(j))/R$. Starting at $p_1(i) = 1/K$, update

$$q_{t+1}(i) = \frac{p_t(i)\exp(-\eta z_{t,i})}{\sum_j p_t(j)\exp(-\eta z_{t,j})}, \tag{16}$$

$$p_{t+1}(i) = (1-\alpha)q_{t+1}(i) + \alpha/K, \tag{17}$$

$$0 < \eta \le 1,\ 0 \le \alpha < 1.$$

Independent draws from each new distribution can waste migrations. Instead, conditional on $A_t = i$, retain i with probability $\min\{1, p_{t+1}(i)/p_t(i)\}$. If it is not retained, sample from the normalized positive residual $(p_{t+1} - p_t)^+$. This maximal coupling has marginal p_{t+1} and switch probability $\mathrm{TV}(p_t, p_{t+1})$. Established dependent-sampling methods motivate this choice [10]; we do not claim it as a new sampling algorithm.

Theorem 9 (Catalogue tracking with migration)**.** *Assume a frozen catalogue, a fixed exogenous sequence of full-information loss vectors, spread at most R, and $M_{ij} \le M$. For any comparator path $u_{1:T}$ with s changes and $0 < \alpha < 1$, the coupled policy satisfies*

$$\mathbb{E}[L_T] - \sum_t \ell_t(u_t) \le \frac{R\,[\log K + s\log(K/\alpha)]}{\eta} + \frac{R\,(T-1-s)\log(1/(1-\alpha))}{\eta} + \frac{R\eta T}{8} + M(T-1)(\eta+\alpha). \tag{18}$$

For $\alpha = 0$ the stationary-comparator bound has numerator $\log K$ and applies only to $s = 0$.

Proof. The fixed-share update is exponential weighting over paths with an initial uniform prior, self-transition mass at least $1-\alpha$, and transition mass at least α/K. The negative log prior of a path with s switches is bounded by the numerator. The bounded exponential-moment inequality gives expected normalized service regret at most that numerator divided by η, plus $\eta T/8$. Since losses are exogenous, the weights are independent of the sampled action path, and the coupling preserves each unconditional marginal. Exponential reweighting changes total variation by at most η; uniform mixing adds at most α. Thus the expected switch count is at most $(T-1)(\eta+\alpha)$. Multiplication by M and addition of service regret proves the bound. □ □

The bound is not a per-run improvement guarantee. Choosing fixed $\eta = .5$ and $\alpha = .01$, as in our frozen experiment, need not make this bound sublinear with increasing T.

Appendix D
Block-Backbone Compiler: Retained Ablation

The exact compiler's cubic cost makes frequent full recompilation inappropriate. For a supplied display order, divide the leaves into consecutive blocks of at most s leaves, and fix a compact b-ary macro-tree on those blocks. Optimize each block's microtree using its restricted completion waves and strict-epoch initialization cost. Replace each macro leaf by its optimized microtree. No evaluation waves enter this process.

The proposition below is exact only under its stated restrictions, and one restriction deserves emphasis because it is easy to misread as a general claim: *every update wave that intersects a macro node's span must cover whole blocks within it.* If a wave can hit a partial block—some but not all of the leaves a macro node covers—then that node's activation weight varies within the class and the objective no longer separates into per-block subproblems; the $O(bns^2)$ claim then fails and the decomposition is only a heuristic. The frozen streams of this paper never emit partial-block waves (a wave's gate set is a union of whole invalidation sets, and the measured waves intersect macro spans block-completely), which is why the decomposition is exact on them; a deployment with partial-block waves must either re-block or fall back to the full exact compiler.

Proposition 10 (Exactness in the block-backbone class)**.** *With display order, block boundaries and macro-tree fixed, independent exact optimization of the blocks minimizes the training objective over every layout in that restricted class. The dynamic-programming work is $O(bns^2)$ with $O(bs^2)$ reusable DP workspace, excluding order learning and wave preparation.*

Proof. Every macro node covers a fixed union of complete blocks, so its activation weight and packet price are constant across the class. All variable costs lie inside disjoint microtrees. The objective is a constant plus the sum of their independent objectives. Conditional exactness therefore gives the minimizing member. There are at most $\lceil n/s \rceil$ subproblems, each of size at most s, yielding the stated DP

bound. Reusing the subproblem arrays yields the workspace bound. □ □

For the reference implementation, M recorded waves and L total wave memberships add filtering and weight-preparation work of $O(\lceil n/s \rceil L + Mns)$, and storage for the input, output and local $M \times s$ arrays. This is not a linear-time claim for the entire pipeline. No bounded approximation ratio against the unrestricted exact tree is asserted; held-out cost can deteriorate. The historical experiment froze $s = 32$ before measurement. Its negative result—-block DP lost to same-order balanced fixed8 in held-out bytes—-motivates excluding this compiler from the practical path.

Appendix E
Information Separations

These results concern the information available to a selector. They are not hardness results for all adaptive tree algorithms, and they do not imply that a complete hypergraph must be recovered: interval hit statistics are sufficient for a fixed order and objective.

A. Marginals do not determine a co-invalidation optimum

Consider four ordered leaves, binary trees, and a constant internal packet price $c > 0$. Let $\mathcal{D}_A$ choose $\{0,1\}$ or $\{2,3\}$ equiprobably and $\mathcal{D}_B$ choose $\{0,3\}$ or $\{1,2\}$ equiprobably. All leaf marginal activation probabilities equal $1/2$ in both distributions. The root and every three-leaf interval are always activated. The five tree shapes fall into the following classes:

Ordered tree	J_A/c	J_B/c
$((0,1),(2,3))$	2	3
$(0,((1,2),3))$, $((0,(1,2)),3)$	3	2.5
$(0,(1,(2,3)))$, $(((0,1),2),3)$	2.5	3

Optimal costs are $2c$ and $2.5c$. A deterministic marginal-only selector receives identical input and must choose the same class, incurring at least $c/2$ regret on one distribution. The last class is weakly dominated by the balanced tree. If a randomized selector chooses the balanced tree with probability q, its regrets are at least $(1-q)c$ and $qc/2$. Their maximum is minimized at $q = 2/3$ and equals $c/3$, which is attainable for this instance. A fixed initialization price adds the same $3c$ to all binary shapes and does not alter the separation.

B. Joint invalidation does not determine progressive layout

Every epoch now invalidates all four checks, and execution order is $(0,1,2,3)$ with singleton completion waves. In workload A, check 0 fails, so only its wave is executed. In workload B, check 3 fails, so all four checks execute. Initial invalidation is identical, even jointly. Every binary tree has three internal nodes; its cost in A is $(3 + \mathrm{depth}(0))c$ and in B is $(3 + \sum_i \mathrm{depth}(i))c$. The root-0 comb minimizes A at $4c$ but costs $12c$ in B, whereas the balanced tree minimizes B at $11c$ but costs $5c$ in A. An invalidation-only randomized selector using a root-0 comb with probability p incurs regrets at least $(1-p)c$ and pc, whose maximum is at least $c/2$. A deterministic selector incurs at least c. This proves a separation between stale-set information and schedule-induced completion information; it does not prohibit a selector that observes both.

Appendix F
Exact Conditional Tree Synthesis: Reference Dynamic Program

The practical default of Section VI-B never calls this dynamic program; it is retained as the exact reference the catalogue construction is compared against, and it is invoked only by the historical research compilers.

Let $D(i,j)$ be the optimal subtree cost and $F_k(i,j)$ the optimal cost of an ordered forest with k nonempty children. With infeasible states equal to $+\infty$,

$$D(i,i) = 0, \tag{19}$$

$$F_1(i,j) = D(i,j), \tag{20}$$

$$F_k(i,j) = \min_{i+k-2 \le u < j} \big\{ F_{k-1}(i,u) + D(u+1,j) \big\}, \tag{21}$$

$$D(i,j) = \min_{2 \le k \le \min(b,\, j-i+1)} \big\{ r_\pi(i,j)c(k) + F_k(i,j) \big\}. \tag{22}$$

Theorem 11 (Exactness conditional on order)**.** *For a fixed π, nonnegative interval weights, and the packet model in Equation (5), Equation (22) returns a minimizing feasible tree in $O(bn^3)$ time and $O(bn^2)$ space.*

Proof. Every feasible root has an arity k in the stated range, so the outer minimization ranges over exactly the feasible root arities. Each candidate tree splits into a root and an ordered forest of k subtrees whose positional intervals partition $[i,j]$; conversely every such split yields a feasible tree. The recurrence over F_k explores exactly these splits, and the induction on interval length proves exactness. There are $O(bn^2)$ states, each scanning at most n splits. The apparent $F_1 = D$ dependency is used only on shorter intervals when constructing a forest with at least two components. □ □

This is a specialization of classical alphabetic-tree optimization [6]. The historical three-proposal reference multiplies compilation time by a constant, not by $n!$. Dense spectral computation can itself cost $O(n^3)$; the reference clustering implementation uses $O(n^2 \log n)$ heap operations and $O(n^2)$ storage.

Appendix G
Cost, Statistical, and Online Proof Details

A. Full-refresh lower bound and global negative control

For an n-leaf tree with m internal nodes, the number of edges is $n+m-1$, so $\sum_v d_v = n+m-1$. Therefore

$$W(L) = S(n-1)+(H+S)m \geq S(n-1)+(H+S)\left\lceil \frac{n-1}{b-1} \right\rceil. \tag{23}$$

The inequality follows from $n-1 = \sum_v (d_v - 1) \leq m(b-1)$. At $n = 64$, $b = 8$, a complete eight-child construction attains the bound with $m = 9$ and $W = 27{,}648$ bytes. Thus all selected global-control layouts can tie an analytically minimal full refresh. Strict initialization followed by one global completion wave costs $2W$. This explains the exact negative control without assuming every balanced tree at every n attains the lower bound.

B. Uniform statistical control at a fixed order

For one activation wave per episode and a fixed registry order, let $M_n = n(n+1)/2$ and $\epsilon = \sqrt{\log(2M_n/\delta)/(2m)}$. The interval hit indicators are bounded independent variables across independent episodes, though checks within an episode may be arbitrarily dependent. Hoeffding plus a union bound gives simultaneous interval-frequency error at most ϵ. For every feasible tree, $|\widehat{J}(T) - J(T)| \leq W(T)\epsilon$. Inserting the empirical minimizer gives regret at most $[W(\widehat{T}) + W(T^*)]\epsilon \leq 2(n-1)C\epsilon$. This is the original HACT bound, retained for attribution. It is not automatically valid uniformly over arbitrary data-selected permutations; the separate held-out catalogue bound in Proposition 7 addresses that selection step.

For strict epochs with at most q completion waves, a loose range is $B = (q+1)(n-1)C$. Under the evaluated sequential eight-record flush rule with at most one final record per obligation, $q \leq \lceil n/8 \rceil$. A narrower observed range is not automatically a proved bound for an unobserved deployment distribution.

C. Maximal coupling and horizon dependence

For consecutive distributions p, p', the probability mass that remains at index i under the coupling is $\min(p_i, p'_i)$. The remaining mass is distributed as $(p'-p)^+$, so the new marginal is exactly p' and $\Pr(A_{t+1} \neq A_t) = \mathrm{TV}(p, p')$. For exponential reweighting by $a_i = \exp(-\eta z_i)$ with $z_i \in [0,1]$, the normalizer is at most one, hence $q_i \geq e^{-\eta} p_i$. Thus $\sum_i \min(p_i, q_i) \geq e^{-\eta}$ and $\mathrm{TV}(p,q) \leq 1 - e^{-\eta} \leq \eta$. Mixing q with the uniform distribution by weight α changes total variation by at most α. The triangle inequality gives the switch bound in Theorem 9.

For $K \geq 2$, $s = 0$, and $\alpha = 0$, write $B_T = RT/8 + M(T-1)$. The bound is $R\log K/\eta + \eta B_T$. When $\sqrt{R\log K/B_T} \leq 1$, choosing that learning rate gives at most $2\sqrt{R\log K\, B_T}$ expected regret, sublinear for fixed R, M, K. This is an established style of switching-cost bound, not a new rate. A single empirical switch with estimated future saving d and anticipated horizon h satisfies the familiar break-even inequality $hd > M_{ij}$. This is arithmetic, not a drift theorem: savings may reverse before h events.

Appendix H
Evidence-to-Claim Register

Claim	Evidence	Scope boundary
Exact ordered optimum	Theorem 11; exhaustive enumeration tests	Fixed order and price model
Validation after order	Proposition 7; Hoeffding bound	Fixed catalogue, no shift
Publication soundness	Theorem 8; conditional proof	Trusted checker, protected state
Practical byte reduction	Table VII; frozen seeds	Clustered workloads, 3 seeds
Source replay reductions	Table VIII; 36 replays	Diagnostic, not fresh checks
Root status identical	Equation (12); 593–594 bytes	Fixed tokenizer, layout-independent
Break-even accounting	Equations (14)–(15)	Declared C, E, R, codec-matched
BPE tokenization gap	`cl100k_base` or `o200k_base` remain UNKNOWN; admission tool supplied	Vocabulary retrieval failed; no billing claim
Context-retirement simulator study	Pre-registered, factorial, executed: v2 = 1,176 runs + 63 long-horizon, seven retention levels × scheduler factor, five closures (two proposals rejected by their gates)	Measured in the simulator; live cohort still protocol-only

Appendix I
Retirement-Study Protocol Deviations and Engine History

Two protocol generations were run. Generation v1 (file-frozen 2026-09-23; `docs/protocols/CONTEXT_RETIREMENT_PROTOCOL_v1.md` with amendments v1.1–v1.4) was written before cohort data generation, and each amendment was written before the run it governs. The engine went through four substantive revisions during that pre-data window; the final cohort was generated once, by the amended engine, and never regenerated after any manuscript text was written. For auditability we disclose the full history, including the errors that motivated each amendment:

1) **Amendment v1.1 (pre-data).** The HACT workload generators draw invalidation sets i.i.d.; retirement is

temporal, so persistence transitions (0.6) and fixed obligation windows (six episodes) were added. A single-stream smoke test of the original design exposed nominal (name-based) obligation retention; the amendment made retention emergent from each policy's mechanism.

2) **Amendment v1.2 (pre-data).** A 630-run smoke test to a temporary directory (results void, never cited) exposed three under-specifications: hand-rolled evidence accounting, summary/reset modeled as sleeping between resets rather than attending, and a frontier capsule that did not scale with contents. All were pinned before the final run.
3) **Amendment v1.3 (pre-data).** A single-stream check showed facts carrying no payload, making replay artificially cheap; payloads (lognormal, median 128 B), payload-charged retrieval, and interface-only capsules were pinned.
4) **Amendment v1.4 (pre-data).** Final verification exposed missing dependency footprints (the review's exported interfaces), an inconsistent summary/reset failure model, and the need for retrieval operations as a co-primary outcome. The evidence ablation was refined to a held-out half with a learned layout order and a steel-manned flat ledger taking the better of full and delta exports.
5) **Post-freeze fixes.** A flat-ledger registry-size off-by-one was corrected after the first final-format cohort; the cohort was regenerated once, before any manuscript table was generated from it. One obligation-resolution accounting error (retained-value reads not crediting resolution) was fixed during pre-data engine verification; all obligation flows are now covered by the test suite (`tests/retirement`), which also pins determinism, the closure outcomes, and the per-family evidence directions.
6) **Post-publication correction (v7.1, 2026-09-28; disclosed).** The v7.0 manuscript stated that the 20% promotion gate was "already satisfied at $c = 0$" because $B_f < B_m$ (with $c_{0.8} = 0$ pooled and per family), and Proposition 4 carried a transposed monotonicity direction. Both were sign errors in the *break-even algebra*, not in the cohort: a byte advantage of $B_f < B_m$ with $R(0) = 0.920$ does not clear a $\rho = 0.8$ gate. The corrected thresholds are $c_{0.8} \approx 1{,}364$ B pooled ($\approx 1{,}047$ B clustered, ≈ 191 B independent, never in global), $c_{0.85} \approx 658$ B pooled, and $c_{0.75} \approx 2{,}443$ B pooled. No confirmatory result changes: the frozen null closure's comparator, threshold, and observed 29.6% regime-O reduction at the 4,096 B price are unchanged, as are all sign tests, exposure fractions, and the long-horizon table. What changes is the deployment advice: a pilot must measure its round-trip price rather than assume the byte direction clears the gate. The same error existed in the `breakEven` helper of `exploratory.ts` (its "`num <= 0: gate already met`" branch) and in the E1 text of the pre-specified exploratory protocol; all three are corrected together, and regression tests now pin the corrected semantics against the frozen pooled means.
7) **Major revision (v8.0, 2026-09-28; scoped claims and consistency audit).** An external review of v7.1 recommended major revision: internal consistency as the most urgent issue, no necessary-and-sufficient retirement-rule claim, and no use of the simulator's numerical thresholds as validated deployment guidance. This revision (a) renames the central claim a *conversion heuristic* and re-frames every necessity/sufficiency statement as model-internal (the failure direction is a measured property of the seven frozen arms, not of all conceivable policies; the sufficiency theorem is a statement about the engine's semantics); (b) re-labels all break-even thresholds as properties of the frozen accounting rather than deployment go/no-go rules, and rewrites the deployment section as two tiers—what the simulator measures and what remains protocol-only; (c) re-titles the manuscript; and (d) adds a 25-point numeric consistency audit of every in-text figure against the frozen cohort (all pass), plus uniform simulator/mechanism-level terminology. No data, cohort, closure, or theorem changed in this revision; only claim scope and presentation.

No amendment was written after the final cohort was generated. The v1 cohort, closures, and summary JSON are reproduced by `bun src/harness/retirement/cli.ts` in under two minutes and are byte-identical across runs; the v2 cohort, closures, and tables are reproduced by `bun src/harness/retirement/run2.ts` and `bun src/harness/retirement/tex2.ts` with the same property. Both engines, both protocols, and both frozen cohorts are committed; v2's protocol and engine-notes commits precede its data commit, which precedes the first v2 manuscript-text commit.

Appendix J
Generation-v1 Record (Complete)

The v1 study is superseded by the v2 factorial (Section IV) but is retained in full for provenance: v2's design was chosen to repair v1's weaknesses, and v1's closures motivated v2's frozen rules. The v1 protocol was frozen in files on 2026-09-23, before its cohort ran; it was first committed to git on 2026-09-24, after the v2 protocol commit (333d495). v1's freeze is therefore file-attested, and the ordering is disclosed rather than smoothed.

a) Design.: Ten arms over $3 \times 3 \times 7 = 63$ cells (630 runs, 40 episodes each): full-replay, masking, summary-reset, bounded-hier, verified-board, frontier, frontier+adapt, adapt-only, frontier+flat, frontier+hact. Three closure rules: null (frontier $\leq$80% of best-baseline cost-per-accepted), attribution (frontier+adapt within 10% of adapt-only), HACT (tree $\leq$95% of flat steel-man on held-out evidence bytes).

b) Frozen outcomes (as generated 2026-09-23, never recomputed).:

- *Null closure triggered*: frontier cost-per-accepted 1,906,765 versus best baseline 564,773 (threshold ≤451,818). No byte-cost claim for frontier retirement.
- *Summary/reset falsified*: 21/63 accepted, 30,820 stale reads, 280 lost obligations—the lossy-conversion failure mode.
- *Operations*: frontier cut retrieval operations 48.3% versus masking (27,728 vs 53,583 across the cohort), the observation that became v2's regime-O hypothesis.
- *Attribution closure triggered*: scheduler dimension byte-neutral (frontier+adapt identical to frontier on bytes); read dimension-separated (makespan only; 87.4% fewer rounds with adaptive batching).
- *HACT closure not triggered*: held-out evidence bytes 59.8% lower than the flat steel-man pooled; per-family split (+88.7% clustered, +49.2% independent, −87.1% global) forced the conditional reading formalized in v2's density rule.

c) Why v2 replaced the design.: v1's arms were hand-picked combinations, so factor attribution leaned on pairwise comparisons and a single attribution closure; its null rule read one pooled byte threshold; its capsule accounting was an end-of-run total with no independent audit; and it priced one cost regime, which is why the operations result could only be reported as a hypothesis. Each weakness became a v2 design item (Section IV-B). v1's cohort, closures, and summary remain reproducible byte-identically from the committed v1 engine via `bun src/harness/retirement/cli.ts`.

References

[1] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. https://doi.org/10.1109/32.962562.

[2] Milos Gligoric, Lamyaa Eloussi, and Darko Marinov. Practical regression test selection with dynamic file dependencies. *Proceedings of ISSTA*, 2015.

[3] You Wang, Michael Pradel, and Zhongxin Liu. Names Are All You Need: Effective and Safe Regression Test Selection for Python. arXiv:2605.25356v1, May 2026.

[4] Umut Acar, Guy Blelloch, Matthias Blume, Robert Harper, and Kanat Tangwongsan. A Library for Self-Adjusting Computation. *Electronic Notes in Theoretical Computer Science*, 148:127–154, 2006.

[5] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP), Article 79, 2018.

[6] Hirendu Vaishnav and Massoud Pedram. Alphabetic Trees: Enumeration and Optimization with Applications to VLSI Design. Author-hosted manuscript, Sections 4.1–4.4; publication date not established from the retrieved copy.

[7] Ziheng Shangguan, Aviv Yaish, and Dahlia Malkhi. Authenticated Data Structures for Dynamic Workloads. arXiv:2608.25206v1, August 2026.

[8] Jonathan E. Atkins, Erik G. Boman, and Bruce Hendrickson. A Spectral Algorithm for Seriation and the Consecutive Ones Problem. *SIAM Journal on Computing*, 28(1):297–310, 1998.

[9] Mark Herbster and Manfred K. Warmuth. Tracking the Best Expert. *Machine Learning*, 32:151–178, 1998.

[10] Sascha Geulen, Berthold Vöcking, and Melanie Winkler. Regret Minimization for Online Buffering Problems Using the Weighted Majority Algorithm. *Proceedings of COLT*, pages 132–143, 2010.

[11] Nicolò Cesa-Bianchi, Ofer Dekel, and Ohad Shamir. Online Learning with Switching Costs and Other Adaptive Adversaries. *Advances in Neural Information Processing Systems*, 26, 2013.

[12] Zexun Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104v1, June 2026.

[13] Zihao Zheng, Baichuan Li, Junyi Yao, and Jiayu Long. Engineering Reliable Commit Gates for Agentic AI: Cost-Aware Verification Portfolios under Common-Mode Data Failures. arXiv:2609.10969v1, September 2026.

[14] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*, 2024.

[15] OpenTelemetry contributors. Batch Processor documentation, official collector implementation. https://github.com/open-telemetry/opentelemetry-collector/tree/main/processor/batchprocessor.

[16] Lindenbauer et al. The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. arXiv:2508.21433v3, October 2025.

[17] Sun et al. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967.

[18] Chen et al. Towards a Science of Scaling Agent Systems (v3). Companion review reference [R4]; controlled scaling results depend on task-architecture alignment.

[19] Yuzhen Mao and Azalia Mirhoseini. Decentralized Multi-Agent Systems with Shared Context. arXiv:2606.10662v1, June 2026.

[20] OpenAI contributors. tiktoken: BPE tokenization implementation and encoding registry. https://github.com/openai/tiktoken.

[21] SWE-bench contributors. The Harness: Docker-based patch evaluation reference. https://www.swebench.com/SWE-bench/reference/harness/.

[22] B. Constantine, G. Forget, R. Geib, and R. Schrage. Framework for TCP Throughput Testing. RFC 6349, August 2011.

[23] Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.